

Learning Bilevel Policies over Symbolic World Models for Long-Horizon Planning

Dillon Z. Chen^{1,2,3,4} Till Hofmann⁵ Toryn Q. Klassen^{1,2} Sheila A. McIlraith^{1,2}

¹Vector Institute ²University of Toronto
³LAAS-CNRS ⁴University of Toulouse ⁵RWTH Aachen University

Abstract

We tackle the challenge of building embodied AI agents that can reliably solve long-horizon planning problems. Imitation learning from demonstrations has shown itself to be effective in training robots to solve a diversity of complex tasks requiring fine motor control and manipulation over low-level (LL), continuous environments. Yet, it remains a difficult endeavour to generate long-horizon plans from imitation learning alone. In contrast, high-level (HL), symbolic abstractions facilitate efficient and interpretable long-horizon planning. We propose to combine the strengths of LL imitation learning for manipulation and control, and HL symbolic abstractions for long-horizon planning. We realise this idea via *bilevel policies* of the form (π^{hl}, π^{ll}) , consisting of a neural policy π^{ll} learned from LL demonstrations, and an HL symbolic policy π^{hl} that is constructed from symbolic abstractions of the LL demonstrations combined with inductive generalisation. We implement these ideas in the BISON system. Experiments on extended MetaWorld benchmarks demonstrate that BISON generalises to long horizons and problems with greater numbers of objects than those solved by VLA and end-to-end methods, and is more time and memory efficient in training and inference. Notably, when ignoring LL execution, BISON’s HL policies can solve HL problems with 10,000 relevant objects in under a minute.

Project page: <https://dillonzchen.github.io/bison>

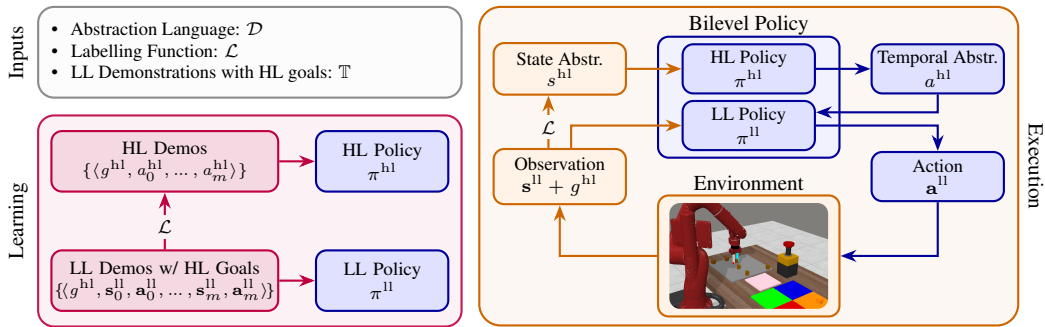


Figure 1: **Top Left** – inputs for learning and executing bilevel policies: a domain theory $\mathcal{D}$, a labelling function $\mathcal{L}$ that maps observations to state abstractions, and LL demos with HL goals. **Bottom Left** – bilevel policy learning: LL demos induce HL demos via $\mathcal{L}$, and LL/HL policies are separately learned from LL/HL demos. **Right** – bilevel policy execution: state abstractions s^{hl} are computed from observations s^{ll} via $\mathcal{L}$ to propose HL actions a^{hl} which in turn help propose LL actions a^{ll} .

1 Introduction

Long-horizon planning remains a longstanding challenge to building embodied AI agents. We present a framework that combines the strengths of symbolic- and neural-based learning and reasoning to produce goal-conditioned policies that address long-horizon planning and acting in continuous state and action spaces with varying numbers of objects.

Scaling up reinforcement learning in simulation and imitation learning from demonstrations has produced robot policies capable of diverse motor control and manipulation tasks [4, 52, 136, 55, 32, 53, 153, 64, 58, 129, 57, 150]. However, scaling alone currently appears insufficient for efficient long-horizon planning and acting [34, 61, 81, 98]. In contrast, symbolic planners (e.g., [50, 22, 109, 111, 11, 123]) over symbolic world models have proven to be highly effective long-horizon planners, but do not easily extend to settings with varying sensory modalities and hard-to-model low-level dynamics. More recently, a suite of bilevel planning systems has emerged that combine symbolic and neural methods (e.g., [124, 71, 39, 116]). Unfortunately, they reflect a number of the inherent shortcomings of symbolic planning as they can struggle with open-world planning and partial observability.

In this work, as depicted in Figure 1, we present a novel approach to long-horizon planning, combining the benefits of low-level (LL) imitation learning with the affordances of high-level (HL), symbolic abstraction, reasoning, and learning. Rather than generating plans from a symbolic domain theory, our approach extracts and inductively generalises symbolic policies from abstractions of our LL demonstrations, and learns to realise those symbolic policies at the LL by means of behaviour cloning.

We realise our approach via the creation of *bilevel policies* of the form $(\pi^{\text{hl}}, \pi^{\text{ll}})$, consisting of a neural policy π^{ll} , learned from multiple LL demonstrations, and an HL symbolic policy π^{hl} that is constructed from a symbolic abstraction of the LL demonstrations. We apply goal regression [135, 88, 103, 140]—a pre-image rewriting technique—to the abstracted symbolic demonstrations to extract a set of *Condition* $\mapsto$ *Action* rules. We inductively generalise the resulting rules to produce compact and expressive HL policies consisting of first-order, condition-action rules that can operate in open-world environments [104, 108, 85]. Our LL policies π^{ll} are represented by a graph neural network [110, 67, 42] conditioned on actions returned by π^{hl} .

We implement our approach in the BISON system. We compare BISON to 8 baselines spanning VLA, end-to-end, and symbolic planning methods on 8 environments extending the MetaWorld benchmarks [147, 93]. Experiments show that BISON solves tasks with longer horizons and more objects than those solved by end-to-end and VLA baselines, and handles more complex environment dynamics than symbolic planning baselines. Our contributions include:

- We introduce BISON, an approach that learns *bilevel policies* $(\pi^{\text{hl}}, \pi^{\text{ll}})$ from LL demonstrations paired with HL goals, given a domain theory $\mathcal{D}$ and labelling function $\mathcal{L}$.
- We apply goal regression and inductive generalisation to learn compact, expressive HL policies π^{hl} that can generalise to arbitrary numbers of objects, and can operate in open-world and partially observable environments, and we learn LL policies via a compact graph neural network π^{ll} with fewer than 33,000 parameters.
- We conduct 21,600 episodes’ worth of experiments across various baselines and environments extending the MetaWorld benchmark. Results demonstrate that BISON achieves superior efficiency, generalisation, and long-horizon planning capabilities compared to baselines. When ignoring LL execution, BISON’s HL policies can solve problems with at least 10,000 objects in under a minute.

2 Related Work

Planning with Abstractions and Hierarchical Reinforcement Learning The idea of leveraging abstractions to facilitate efficient long-horizon planning is a longstanding one [107, 127, 43, 9, 14, 35, 28, 97, 90, 68, 10]. Reinforcement learning (RL) approaches have also looked at learning and leveraging hierarchies and abstractions [27, 49, 126, 29, 77, 71, 2, 1, 74] to improve sample efficiency. There has also been recent interest in leveraging formal languages to represent such abstractions as well as to specify goals for RL agents [78, 130, 17, 15, 56, 133, 131, 134, 101, 143, 59, 75]. Our work is inspired by the use of HL abstractions but differs in the problem setting: we learn policies entirely from demonstrations over both the abstraction and underlying environment that generalise to longer horizon problems than RL and exploration methods.

Task and Motion Planning Task and motion planning (TAMP) is a common framework for solving embodied AI that integrates HL planning and LL execution. TAMP methods can be categorised [39, 151] into interleaved search-then-sample [124, 114, 94, 115] or hybrid constraint satisfaction and optimisation [87, 132, 26] approaches. Later works employ learning in various manners: predicting HL action feasibility [137, 31, 142, 12, 144, 13], learning problem transformations, heuristics or policies for guiding search [19, 63, 119, 25, 62, 89, 94, 21, 33], and learning policies that aim to imitate bilevel planners [30, 92, 152, 82]. Our work falls in the latter category whereby we learn policies that solve bilevel planning problems without relying on search. Our work differentiates itself within this category through learning bilevel policies over both the LL and HL spaces for facilitating efficient, interpretable and long-horizon planning in contrast to purely end-to-end methods. Furthermore, some TAMP search methods struggle with open-world planning and partial observability, owing to their reliance on symbolic planners, while our approach can support these problem settings.

3 Problem Statement

We are interested in learning policies over continuous environments that address HL goals from (1) LL demonstrations paired with HL goals, (2) an abstraction language $\mathcal{D}$, and (3) a labelling function $\mathcal{L}$ that maps LL observations to HL abstractions, as depicted in Figure 1, that can generalise to unseen states and goals in a zero-shot manner. We follow a basic *bilevel planning* paradigm (e.g., [124, 39]) in which a planning problem, consisting of continuous LL states and actions, admits an HL model-based abstraction using symbolic and relational states and actions. We then list assumptions applied in our planning representations, and corresponding related work that validates such assumptions. A summary of definitions, notations and typography semantics is provided in Section A.

Bilevel Planning A *planning problem* is a tuple $\mathbf{P} = \langle \mathbf{S}, \mathbf{A}, s_0, \mathbf{g} \rangle$ where $\mathbf{S}$ is a set of states, $\mathbf{A}$ is a set of actions, s_0 is the initial state and $\mathbf{g} \subseteq \mathbf{S}$ is a set of goal states. An action $\mathbf{a} \in \mathbf{A}$ is a function $\mathbf{a} : \mathbf{S} \rightarrow \Delta(\mathbf{S} \cup \{\perp\})$ that maps to a distribution of possible next states or $\perp$ indicating the action is not applicable. A policy is a distribution over actions $\pi(\mathbf{a} \mid \mathbf{s}, *)$ conditioned on a state and other arguments $*$. A policy is a *solution* for a planning problem if it achieves a goal state from the initial state s_0 with an expected probability of 1. A *bilevel planning problem* is a tuple $\mathbb{P} = \langle \mathbf{P}^{\text{ll}}, \mathbf{P}^{\text{hl}}, \mathcal{L} \rangle$ consisting of an HL and LL planning problem and a *labelling function* $\mathcal{L} : \mathbf{S}^{\text{ll}} \rightarrow \mathbf{S}^{\text{hl}}$ such that $\mathbf{g}^{\text{hl}} = \{\mathcal{L}(s^{\text{ll}}) \mid s^{\text{ll}} \in \mathbf{g}^{\text{ll}}\}$. We next depict conditions on $\mathcal{L}$ for ensuring useful HL abstractions of LL problems, followed by representations of HL and LL planning problems for our work.

Downward Refinement Property We next introduce a property of our bilevel planning paradigm that establishes when the HL abstraction is useful for constructing bilevel planning solutions. The downward refinement property (DRP) [9] states that any solution for an HL problem can be refined into a solution for the LL problem. We extend this notion to the general nondeterministic setting. Given a state $\mathbf{s}$ and policy π , we denote $\text{exec}(\mathbf{s}, \pi)$ to be the set of all possible successor states achievable by applying some action with non-zero support in $\pi(\cdot \mid \mathbf{s})$.

Definition 1 (Nondeterministic Downward Refinement Property). A *bilevel planning problem* $\langle \mathbf{P}^{\text{ll}}, \mathbf{P}^{\text{hl}}, \mathcal{L} \rangle$ exhibits the nondeterministic downward property (NDRP) if for every π^{hl} that solves $\mathbf{P}^{\text{hl}}$, there exists π^{ll} that solves $\mathbf{P}^{\text{ll}}$ where for every s^{ll} reachable by π^{ll} in $\mathbf{P}^{\text{ll}}$,

$$\forall \mathbf{s} \in \text{exec}(s^{\text{ll}}, \pi^{\text{ll}}), \mathcal{L}(\mathbf{s}) \in \text{exec}(\mathcal{L}(s^{\text{ll}}), \pi^{\text{hl}}) \cup \{\mathcal{L}(s^{\text{ll}})\}. \quad (1)$$

Intuitively, the LL policy can be viewed as analogous to a set of skills or options [69] for implementing and refining HL actions. NDRP states that every step determined by π^{ll} either stays within the same HL abstract state or advances to a valid HL successor under π^{hl} . In this way, a solution for the HL problem can be used as guidance to build a solution for the underlying LL problem.

LL Representation: Object-Centric, Ego-Centric Viewpoints We represent states in our LL problems $\mathbf{P}^{\text{ll}}$ in an object- and ego-centric manner. This means that states are represented as tuples $s^{\text{ll}} = \langle \mathbf{x}_e, \{\mathbf{x}_o\}_{o \in \mathbf{O}} \rangle$ where $\mathbf{x}_e \in \mathbb{R}^n$ is a vector representing the agent’s state (e.g., location of a robot), $\mathbf{O}$ is a set of objects visible to the agent, and $\{\mathbf{x}_o\}_{o \in \mathbf{O}}$ where $\mathbf{x}_o \in \mathbb{R}^m$ is a set of vectors representing the states (e.g. position) of all other objects in the world relative to the agent. LL actions are black box functions that determine transitions between LL states with unknown probabilities. The object-centric representation is commonly used for embodied AI planning [118, 120, 73, 79, 116] for which there is much work for learning or extracting such representations from raw sensory data [16, 5, 66, 149, 60, 148, 47, 138, 99].

HL Representation: Relational State and Action Abstractions We represent our HL problems $\mathbf{P}^{\text{hl}}$ via state and temporal abstractions of the world in a relational form, inspired by the STRIPS [37] and PDDL languages [91, 41, 40, 48] from symbolic planning. Specifically, LL states can be abstracted into HL states represented as symbolic databases and LL actions can be temporally abstracted into HL actions via an action schema language. We define a planning *domain* as a tuple $\mathcal{D} = \langle \mathcal{P}, \mathcal{A} \rangle$ consisting of a set of predicates $\mathcal{P}$ and action schemata $\mathcal{A}$.

A *predicate* is a symbol p with a set of argument terms, denoted $p(x_1, \dots, x_{n_p})$ where $n_p \in \mathbb{N}_0$ depends on p and denotes the arity of p . A fact is a predicate whose argument terms are all instantiated with objects denoted $p(o_1, \dots, o_{n_p})$, and an HL state is a set of facts. We concern ourselves with *open-world planning* where facts that are not present in the state are assumed unknown [44], in contrast to the more restrictive closed-world planning exhibited by PDDL where facts not present in the state are assumed false [37]. Our labelling function $\mathcal{L}$ maps from LL to HL states with predicates from $\mathcal{D}$. We assume that all LL goal states can be abstracted into a single HL *goal condition* g^{hl} defined as a set of facts. An HL state s^{hl} is a goal state if $g^{\text{hl}} \subseteq s^{\text{hl}}$.

An *action schema* is a temporally-abstracted, nondeterministic action that can be parameterised by objects, given by a tuple $\langle \text{var}(a), \text{pre}(a), \{\text{add}_i(a), \text{del}_i(a)\}_{i=1}^n \rangle$ where $\text{var}(a)$ is a set of parameter variables, and the preconditions $\text{pre}(a)$ and nondeterministic add $\text{add}_i(a)$ and delete $\text{del}_i(a)$ effects are sets of predicates with arguments instantiated with variables from $\text{var}(a)$. An HL action a^{hl} is an action schema a where each variable is instantiated with an object, denoted $a^{\text{hl}} = a(o_1, \dots, o_{n_a})$, and is applicable in an HL state s^{hl} if $\text{pre}(a^{\text{hl}}) \subseteq s^{\text{hl}}$ in which case we define its successor states

$$\text{succ}(s^{\text{hl}}, a^{\text{hl}}) = \{(s^{\text{hl}} \setminus \text{del}_i(a^{\text{hl}})) \cup \text{add}_i(a^{\text{hl}})\}_{i=1}^n. \quad (2)$$

Example 1 (Pick and Place Domain). We can model a simple pick and place domain $\mathcal{D} = \langle \mathcal{P}, \mathcal{A} \rangle$ where a robot can transport items between locations with $\mathcal{P} = \{rAt(x), at(x, y), \text{free}(), \text{hold}(x)\}$ and $\mathcal{A}$ containing the three deterministic action schemata:

$$\begin{array}{lll} \text{pick}(o, l) : \text{pre} : \{rAt(l), at(o, l), \text{free}()\} & \text{move}(l1, l2) : \text{pre} : \{rAt(l1)\} & \text{place}(o, l) : \text{pre} : \{rAt(l), \text{hold}(o)\} \\ \text{add} : \{\text{hold}(o)\} & \text{add} : \{rAt(l2)\} & \text{add} : \{at(o, l), \text{free}()\} \\ \text{del} : \{at(o, l), \text{free}()\} & \text{del} : \{rAt(l1)\} & \text{del} : \{\text{hold}(o)\} \end{array}$$

Where do $\mathcal{D}$ and $\mathcal{L}$ come from? There is significant work in generating HL state and temporal abstractions in this form from raw sensory data (e.g., [70, 71, 6, 7, 20, 121, 60, 118, 120, 113, 139, 54, 76]), whose progress has been propelled [84, 46, 8, 3, 79, 76, 145, 116, 80] by vision language model advances [102, 128]. The result is an HL domain description $\mathcal{D}$ along with a labelling function $\mathcal{L}$ that maps LL states to HL states. Specifically, $\mathcal{D}$ may be manually generated by a human and that $\mathcal{D}$ and $\mathcal{L}$ can be generated via a VLA or learned.

Problem Statement Summary We can now define the problem: given an HL domain $\mathcal{D}$ that describes the abstract actions available to the agent, a labelling function $\mathcal{L}$ that maps LL states to HL states, and a finite dataset $\mathbb{T} = \{\langle g_i^{\text{hl}}, s_0^{\text{ll}}, \mathbf{a}_0^{\text{ll}}, \dots, s_{m_i}^{\text{ll}}, \mathbf{a}_{m_i}^{\text{ll}} \rangle\}_{i=1}^n$ consisting of LL state-action demonstrations along with HL goals, our objective is to learn a goal-conditioned policy $\pi(\mathbf{a}^{\text{ll}} \mid s^{\text{ll}}, g^{\text{hl}})$ conditioned on LL states and HL goals that chooses low-level actions to achieve the goal.

4 Learning Bilevel Policies for Bilevel Planning

We learn bilevel policies from the given HL-goal labelled LL demonstrations $\mathbb{T}$, using the HL domain $\mathcal{D}$ and labelling function $\mathcal{L}$. A *bilevel policy* (BP) consists of a pair of policies $(\pi^{\text{hl}}, \pi^{\text{ll}})$ of the forms $\pi^{\text{hl}}(a^{\text{hl}} \mid s^{\text{hl}}, g^{\text{hl}})$ and $\pi^{\text{ll}}(\mathbf{a}^{\text{ll}} \mid s^{\text{ll}}, a^{\text{hl}}, g^{\text{hl}})$, respectively. Both policies are goal-conditioned and can be reused across different problems. Together with the labelling function $\mathcal{L}$, the pair of policies can be composed into a single policy operating on the LL environment conditioned on HL goals:

$$\pi(\mathbf{a}^{\text{ll}} \mid s^{\text{ll}}, g^{\text{hl}}) = \sum_{a^{\text{hl}}} \pi^{\text{ll}}(\mathbf{a}^{\text{ll}} \mid s^{\text{ll}}, a^{\text{hl}}, g^{\text{hl}}) \cdot \pi^{\text{hl}}(a^{\text{hl}} \mid \mathcal{L}(s^{\text{ll}}), g^{\text{hl}}). \quad (3)$$

In the BP framework, the HL and LL policies are learned separately, with the main idea being that HL policies π^{hl} capture HL decision making, while LL policies π^{ll} provide LL realisations of the HL actions prescribed by π^{hl} . While π^{ll} is learned directly from $\mathbb{T}$ via imitation learning, the HL policy is learned by first constructing an abstract dataset T^{hl} by applying $\mathcal{L}$ to $\mathbb{T}$, and then inductively learning a symbolic policy π^{hl} from T^{hl} . We realise the HL policy π^{hl} as a set of first-order, condition-action rules (Section 4.1) and the LL policy π^{ll} as a graph neural network (Section 4.2).

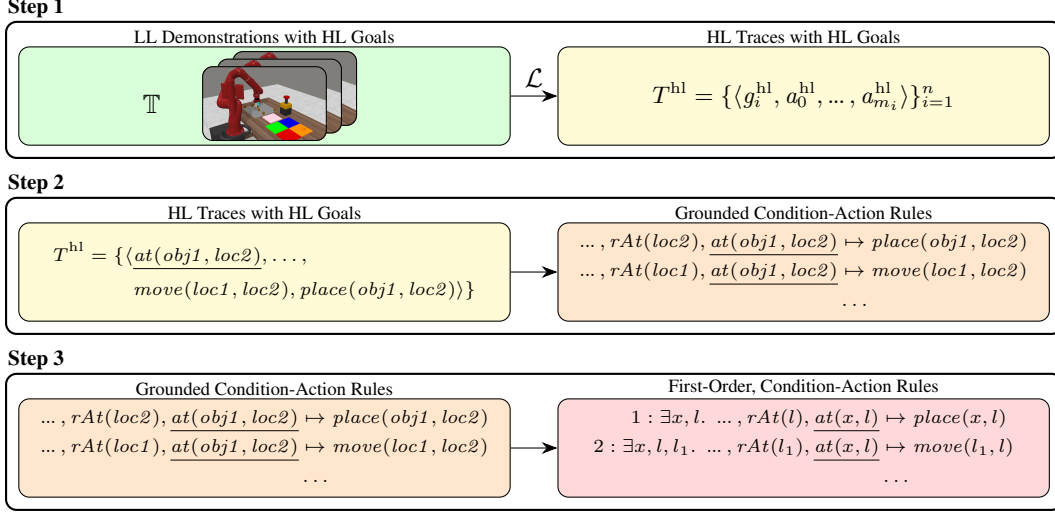


Figure 2: The HL policy $\pi^{\text{hl}}(a^{\text{hl}} | s^{\text{hl}}, g^{\text{hl}})$ learning process. **Step 1:** we use the labelling function $\mathcal{L}$ to construct HL traces from the LL demonstrations paired with HL goals. **Step 2:** we utilise goal regression to extract condition-action rules from the HL traces and goals (underlined). **Step 3:** we inductively generalise the rules by replacing objects with variables to produce symbolic policies.

4.1 Learning HL Policies via Goal Regression and Inductive Generalisation

We construct HL policies $\pi^{\text{hl}}(a^{\text{hl}} | s^{\text{hl}}, g^{\text{hl}})$ from LL demonstrations paired with HL goals in 3 steps illustrated in Figure 2: (1) we first generate HL traces that are abstractions of our LL demonstrations; (2) we then utilise goal regression to extract a set of condition-action rules from such HL traces; and (3) we finally utilise inductive generalisation to produce interpretable symbolic policies in the form of first-order, condition-action rules. Steps 2 and 3 are achieved in milliseconds. In what follows, we first define the form of our HL policies and a means of grounding them for execution with respect to a particular problem instance. We then elaborate on the 3 steps required to generate HL policies.

HL policy representation Our HL policies π^{hl} consist of sets of first-order, condition-action rules r with associated priority values related to goal proximity. These first-order, condition-action rules are tuples of the form $\langle val(r), var(r), sCond(r), gCond(r), action(r) \rangle$ where $val(r)$ denotes the rule priority, $var(r)$ denotes a set of variables, $sCond(r)$ and $gCond(r)$ are first-order conjunctive formulas over predicates in $\mathcal{P}$ representing the state and goal conditions, and $action(r)$ is an action schema denoting the action to take if the conditions hold. Example 2 illustrates such an HL policy.

Example 2. A first-order, condition-action policy for the pick and place domain in Example 1, where goal conditions $gCond$ are indicated by underlining:

$$\begin{array}{ll}
 \begin{array}{c}
 \text{val}(r) \quad \text{var}(r) \quad \quad \quad \text{sCond}(r) \quad \quad \quad \text{gCond}(r) \quad \quad \quad \text{action}(r) \\
 1: \underbrace{\exists x, l. \text{hold}(x)}_{\text{sCond}(r)}, \underbrace{rAt(l)}_{\text{sCond}(r)}, \underbrace{\underline{at(x, l)}}_{\text{gCond}(r)} \mapsto \underbrace{\underline{place(x, l)}}_{\text{action}(r)}
 \end{array} & 3: \exists x, l, l_1. \text{at}(x, l_1), \text{free}(), rAt(l_1), \underline{at(x, l)} \mapsto \underline{pick(x, l_1)} \\
 2: \exists x, l, l_1. \text{hold}(x), rAt(l_1), \underline{at(x, l)} \mapsto \underline{move(l_1, l)} & 4: \exists x, l, l_1, l_2. \text{at}(x, l_1), \text{free}(), rAt(l_2), \underline{at(x, l)} \mapsto \underline{move(l_2, l_1)}
 \end{array}$$

In order to select the next action according to an HL policy π^{hl} , variables within the rules must be replaced with appropriate objects. Given a rule r of π^{hl} , a *grounding* of r is an assignment of the variables $var(r)$ in $sCond(r)$, $gCond(r)$, and $action(r)$ with objects from $\mathcal{O}$. A ground rule $r \in \text{ground}(\pi^{\text{hl}})$ is *applicable* in an HL state s^{hl} with goal g^{hl} if $sCond(r) \subseteq s^{\text{hl}}$ (i.e., the state condition is satisfied by the state) and $gCond(r) \subseteq g^{\text{hl}} \setminus s^{\text{hl}}$ (i.e., the goal condition is satisfied by the set of unachieved goal facts). A policy π^{hl} is executed by finding, via database query algorithms, a ground rule r with lowest $val(r)$ value and ties broken arbitrarily that is applicable in s^{hl} and g^{hl} . Implicitly, this defines a probability distribution $\pi^{\text{hl}}(a^{\text{hl}} | s^{\text{hl}}, g^{\text{hl}})$ over the grounded HL actions, where $\pi^{\text{hl}}(a^{\text{hl}} | s^{\text{hl}}, g^{\text{hl}}) = 1$ if a^{hl} is selected by π^{hl} in state s^{hl} given goal g^{hl} , and 0 otherwise.

Step 1: construct HL traces From an LL trace $\langle g_i^{\text{hl}}, s_0^{\text{ll}}, a_0^{\text{ll}}, \dots, s_m^{\text{ll}}, a_m^{\text{ll}} \rangle \in \mathbb{T}$, the domain $\mathcal{D}$, and the labelling function $\mathcal{L}$, we can extract an HL trace $\langle g_i^{\text{hl}}, a_0^{\text{hl}}, \dots, a_n^{\text{hl}} \rangle$ as follows: starting with the HL

abstraction $s_0^{\text{hl}} = \mathcal{L}(s_0^{\text{ll}})$ of the initial LL state s_0^{ll} , we iteratively determine the LL state where the HL abstraction changes, i.e., where $\mathcal{L}(s_j^{\text{ll}}) \neq \mathcal{L}(s_{j-1}^{\text{ll}})$, which defines the next HL state $s_i^{\text{hl}} = \mathcal{L}(s_j^{\text{ll}})$. We then find an applicable HL action a_i^{hl} which causes the change from s_{i-1}^{hl} to s_i^{hl} , i.e., an a_i^{hl} such that $\text{pre}(a_i^{\text{hl}}) \subseteq s_{i-1}^{\text{hl}}$ and for some effect j , we have $s_i^{\text{hl}} = (s_{i-1}^{\text{hl}} \cup \text{add}_j(a_i^{\text{hl}})) \setminus \text{del}_j(a_i^{\text{hl}})$. By applying this extraction for each LL trace from $\mathbb{T}$, we obtain a HL dataset $T^{\text{hl}} = \{\langle g_i^{\text{hl}}, a_0^{\text{hl}}, \dots, a_m^{\text{hl}} \rangle\}_{i=1}^n$.

Step 2: extract rules Our method converts HL action traces into ground condition-action rules via *goal regression* [36, 135, 88, 103, 104, 38]. Goal regression computes the minimal condition under which an action achieves a goal; when applied recursively, it determines under which condition an action trace will lead to a goal state. Formally, a set of facts g^{hl} is regressable over an HL action a^{hl} if for every nondeterministic outcome i of a^{hl} we have $\text{del}_i(a^{\text{hl}}) \cap g^{\text{hl}} = \emptyset$, in which case we define

$$\text{regr}(g^{\text{hl}}, a^{\text{hl}}) = \{(g^{\text{hl}} \setminus \text{add}_i(a^{\text{hl}})) \cup \text{pre}(a^{\text{hl}}) \mid i = 1, \dots, n\}, \quad (4)$$

and $\text{regr}(g^{\text{hl}}, a^{\text{hl}}) = \emptyset$ otherwise. Notably, goal regression is also a powerful tool for enabling open-world and generalised planning [45, 108, 85].

Step 3: inductive generalisation Finally, we *lift* the (ground) condition-action rules to obtain first-order rules by replacing ground objects with fresh variables. The resulting rules are abstracted from specific problem instances and thus generalise to arbitrary numbers of objects. Given an HL action a^{hl} and sets of facts s^{hl} and g^{hl} , let $o_1, \dots, o_q$ be the union of all objects from the action and facts. We then define the set of variables $\text{var} = \{v_1, \dots, v_q\}$ and mapping $g : \mathbf{O} \rightarrow \text{var}$ defined by $g(o_i) = v_i$. Then we define the lifting operator as

$$\text{lift}(a^{\text{hl}}, s^{\text{hl}}, g^{\text{hl}}) = \langle \text{var}, s\text{Cond}, g\text{Cond}, \text{action} \rangle \quad (5)$$

where $s\text{Cond}$ and $g\text{Cond}$ are the conjunctions of the facts in s^{hl} and g^{hl} respectively with g applied to their objects, and action is the action schema of a^{hl} with g applied on its objects.

Putting the steps together Algorithm 1 describes the procedure of learning an HL policy π^{hl} from LL demonstrations $\mathbb{T}$ via goal regression. It begins by initialising an empty policy (Algorithm 1) and then iterates over the HL trajectories in T^{hl} (Algorithm 1) constructed from the LL demonstrations (Algorithm 1). For each trajectory, it initialises S to be the singleton set containing the goal to be regressed (Algorithm 1). Then it iterates over the HL actions in reverse order (Algorithm 1), and at each step regresses the current goals in S (Algorithm 1) over the current HL action a_j^{hl} to obtain the next set of subgoals S' (Algorithm 1). The actions and regressed goals in each step j are lifted into first-order rules r (Algorithm 1) alongside $g^{\text{hl}'}$, the subset of g^{hl} that is achieved by the suffix of determinised actions from the trajectory (Algorithm 1). The rules are added to the policy π^{hl} with priority $m - j$ (related to goal proximity) (Algorithm 1) and the resulting policy π^{hl} is returned in Algorithm 1.

Generalisation to arbitrarily many objects The following theorem establishes the conditions under which Algorithm 1 returns policies that can generalise to problems with arbitrarily many objects and hence arbitrarily long horizons. The main conditions are that the problems (a) exhibit goals whose components can be achieved independently in any order, as is the case for a large proportion of real-world problems [122, 72], and (b) the goal components can be achieved with bounded size HL policies. These conditions are formally defined in Section B alongside the proof of the theorem. Under these conditions, the proof follows by representing an infinite set S of arbitrarily large HL problems as a finite collection of subproblems equivalent up to object renaming. Each subproblem class exhibits a finite policy and we can compose all such policies into a single, finite policy that can solve every problem in S .

Algorithm 1: HL policy learning

Input: LL demonstrations with HL goals $\mathbb{T}$.

Output: HL policy π^{hl} .

// **Step 1: Construct HL Traces**

```

1  $T^{\text{hl}} \leftarrow \text{constructFrom}(\mathbb{T})$ 
2  $\pi^{\text{hl}} \leftarrow \emptyset$ 
3 for  $\langle g^{\text{hl}}, a_0^{\text{hl}}, \dots, a_m^{\text{hl}} \rangle \in T^{\text{hl}}$ 
4    $S \leftarrow \{g^{\text{hl}}\}$ 
5   for  $j = m, \dots, 0$ 
6      $S^{\text{next}} \leftarrow \emptyset$ 
7     for  $s^{\text{hl}} \in S$ 
8       // Step 2: Extract Rules
9        $S' \leftarrow \text{regr}(s^{\text{hl}}, a_j^{\text{hl}})$ 
10      for  $s^{\text{hl}'} \in S'$ 
11         $g^{\text{hl}'} \leftarrow g^{\text{hl}} \cap \text{succ}(s^{\text{hl}'}, \langle a_j^{\text{hl}'}, \dots, a_m^{\text{hl}'} \rangle)$ 
12        // Step 3: Inductive Generalisation
13         $r \leftarrow \text{lift}(a_j^{\text{hl}}, s^{\text{hl}'}, g^{\text{hl}'})$ 
14         $\pi^{\text{hl}} \leftarrow \pi^{\text{hl}} \cup \{ \langle r, m - j \rangle \}$ 
15       $S^{\text{next}} \leftarrow S^{\text{next}} \cup S'$ 
16    $S \leftarrow S^{\text{next}}$ 
17 return  $\pi^{\text{hl}}$ 
```

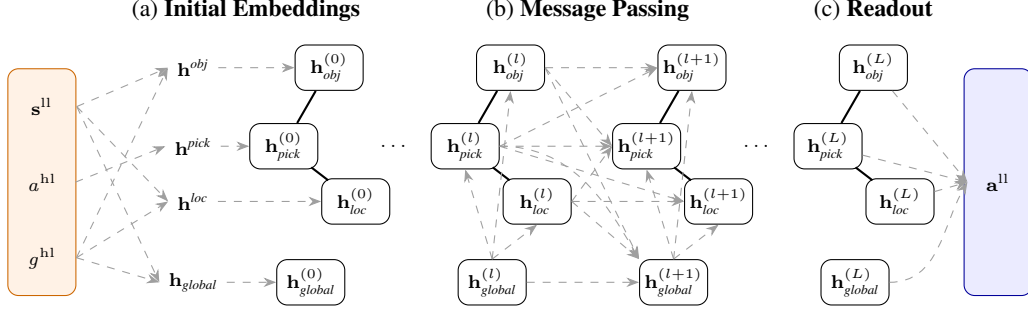


Figure 3: The LL policy $\pi^{\text{ll}}(\mathbf{a}^{\text{ll}} \mid \mathbf{s}^{\text{ll}}, \mathbf{a}^{\text{hl}}, \mathbf{g}^{\text{hl}})$ represented by a GNN. In this example, the input action is $a^{\text{hl}} = \text{pick}(\text{obj}, \text{loc})$, and the resulting output is $\mathbf{a}^{\text{ll}}$. Solid lines represent graph edges, and dashed lines represent how information is passed. **Bold font** indicates Euclidean vectors.

Theorem 1. Let $\mathcal{D} = \langle \mathcal{P}, \mathcal{A} \rangle$ be an HL domain, $\mathcal{L}$ a labelling function, and $C \in \mathbb{N}$. There exists a finite dataset $\mathbb{T}$ such that the HL policy learned from $\mathbb{T}$ via Algorithm 1 solves any HL planning problem $\mathbf{P}^{\text{hl}}$ conforming to $\mathcal{D}$ and satisfying C -bounded goal independence.

4.2 Learning LL Policies via Graph Neural Networks and Imitation Learning

We next realise our LL policies $\pi^{\text{ll}}(\mathbf{a}^{\text{ll}} \mid \mathbf{s}^{\text{ll}}, \mathbf{a}^{\text{hl}}, \mathbf{g}^{\text{hl}})$ with a graph neural network (GNN) architecture.

LL policy representation We describe how to construct a GNN model from $\mathbf{s}^{\text{ll}}$, $\mathbf{a}^{\text{hl}}$ and $\mathbf{g}^{\text{hl}}$, and a domain $\mathcal{D} = \langle \mathcal{P}, \mathcal{A} \rangle$ that generates an embedding for predicting the LL action $\mathbf{a}^{\text{ll}}$. Given vectors $\mathbf{h}_1 \in \mathbb{R}^n$, $\mathbf{h}_2 \in \mathbb{R}^m$, we denote their concatenation as $\mathbf{h}_1 \parallel \mathbf{h}_2 \in \mathbb{R}^{n+m}$. Given $i, n \in \mathbb{N}$, we denote the one-hot encoding $\mathbf{e}_i^n \in \mathbb{R}^n$ with all 0's except 1 in the i -th element. We assume an enumeration of predicates and schemata, and let M be the maximum arity of the schemata (i.e., $\max_{a \in \mathcal{A}} |\text{var}(a)|$).

Let $\mathbf{a}^{\text{hl}} = a(o_1, \dots, o_n)$, $\mathbf{s}^{\text{ll}} = \langle \mathbf{x}_e, \{\mathbf{x}_o\}_{o \in \mathcal{O}} \rangle$ and $\mathbf{g}^{\text{hl}} = \mathcal{L}(\mathbf{s}^{\text{ll}})$. The GNN inputs are initially encoded as a set of embeddings $\mathbf{h}_{\text{global}}, \mathbf{h}_a, \mathbf{h}_{o_1}, \dots, \mathbf{h}_{o_n}$ representing a global node, an action node, and object nodes, respectively, where $\mathbf{h}_{\text{global}} = \mathbf{x}_e \parallel \sum_{p() \in \mathbf{s}^{\text{hl}}} \mathbf{e}_p^{|\mathcal{P}|} \parallel \sum_{p() \in \mathbf{g}^{\text{hl}}} \mathbf{e}_p^{|\mathcal{P}|}$ encodes the agent's state and nullary facts that are true in the HL state and goal, $\mathbf{h}_a = \mathbf{e}_a^{|\mathcal{A}|}$ encodes the action schema, and $\mathbf{h}_{o_i} = \mathbf{x}_{o_i} \parallel \sum_{p(o_i) \in \mathbf{s}^{\text{hl}}} \mathbf{e}_p^{|\mathcal{P}|} \parallel \sum_{p(o_i) \in \mathbf{g}^{\text{hl}}} \mathbf{e}_p^{|\mathcal{P}|} \parallel \mathbf{e}_i^M$ encodes an object o_i 's state, unary facts instantiated with o_i in the HL state and goal, and its position in the action $\mathbf{a}^{\text{hl}}$.

The GNN algorithm then consists of the following three operations, as illustrated in Figure 3. The $\mathbf{W}$ symbols denote learnable weights.

- The initial encodings are embedded: $\mathbf{h}_{\text{global}}^{(0)} = \mathbf{W}_g^{(0)} \mathbf{h}_{\text{global}}$, $\mathbf{h}_a^{(0)} = \mathbf{W}_a^{(0)} \mathbf{h}_a$, $\mathbf{h}_{o_i}^{(0)} = \mathbf{W}_o^{(0)} \mathbf{h}_{o_i}$.
- Next, L rounds of message passing are performed. Object embeddings are aggregated via element-wise max $\mathbf{h}_o^{(l)} = \max_{i=1, \dots, n} \mathbf{h}_{o_i}^{(l)}$ and node embeddings are updated by $\mathbf{h}_{\text{global}}^{(l+1)} = \sigma(\mathbf{W}_g^{(l)} (\mathbf{h}_{\text{global}}^{(l)} + \mathbf{h}_a^{(l)} + \mathbf{h}_o^{(l)}))$, $\mathbf{h}_a^{(l+1)} = \sigma(\mathbf{W}_a^{(l)} (\mathbf{h}_{\text{global}}^{(l+1)} + \mathbf{h}_a^{(l)} + \mathbf{h}_o^{(l)}))$, and $\mathbf{h}_{o_i}^{(l+1)} = \sigma(\mathbf{W}_o^{(l)} (\mathbf{h}_{\text{global}}^{(l+1)} + \mathbf{h}_a^{(l)} + \mathbf{h}_{o_i}^{(l)}))$ for nonlinearity σ .
- Finally, a feed-forward readout layer is then applied to the final embeddings $\mathbf{h}_{\text{global}}^{(L)} + \mathbf{h}_a^{(L)} + \mathbf{h}_o^{(L)}$ to predict the LL action $\mathbf{a}^{\text{ll}}$.

Learning method Following the HL learning method, we can compute a single dataset consisting of tuples $\langle \mathbf{s}^{\text{ll}}, \mathbf{a}^{\text{hl}}, \mathbf{g}^{\text{hl}}, \mathbf{a}^{\text{ll}} \rangle$ from the LL demonstrations corresponding to inputs and outputs of π^{ll} . We optimise the GNN to minimise the mean squared error (MSE) between the predicted LL actions and the ground truth LL actions in the dataset $\mathbb{T}$. In place of MSE optimisation, we could alternatively learn a generative diffusion policy [51, 18]. We leave exploration of this alternative to future work.

5 Experiments

We implement our approach from Section 4 in a system called BISON and conduct experiments on extended MetaWorld [147, 93] benchmarks. We compare our approach to various VLA, end-to-end, and planning systems to demonstrate the efficacy of BISON for learning bilevel policies that can generalise to longer horizons and more objects than seen in its training demonstrations.

Setup We evaluate 7 imitation learning architectures and 2 VLA baselines across 8 MuJoCo robot simulation environments. For each approach and environment, we train 3 models on 200 goal-achieving trajectories from problems with 3 objects. We train the GNN LL policy in Section 4.2 with hyperparameters specified in Section C.1. Each trained policy is evaluated on problems with 1 to 10 objects from each environment with 10 episodes for each object and at most $2048n$ steps for each episode where n is the number of objects. As a result, we conduct up to $9 \times 8 \times 3 \times 10 \times 10 = 21,600$ episodes’ worth of evaluations. Training is performed with an NVIDIA L40 GPU (48GB memory) while evaluation is performed on Intel Xeon Gold 6338 CPUs (2.00 GHz).

Environments We evaluate the aforementioned methods on 8 environments that compositionally extend the tasks from the MetaWorld benchmark suite [147, 93]. There are 4 stationary environments (Blocks^S, Factory^S, Colour^S, Gacha^S) each with a corresponding variant (Blocks^N, Factory^N, Colour^N, Gacha^N) where objects may randomly teleport around throughout the episode. Additional environment semantics and visualisations are provided in Section C.3. Each environment induces a problem distribution with different initial states and goals, parameterised by the number of objects. Table 1 summarises the environments and their uncertainty attributes: **exogenous** transition uncertainty that is not modelled in the HL abstraction, **endogenous** transition uncertainty that is modelled in the HL abstraction, and **state** uncertainty and partial observability of object attributes and locations.

Table 1: Environments and their uncertainty attributes.

Uncertainty	Blocks ^S	Blocks ^N	Colour ^S	Colour ^N	Factory ^S	Factory ^N	Gacha ^S	Gacha ^N
Exogenous		✓						
Endogenous			✓	✓				
State					✓	✓	✓	✓

Methods We evaluate and compare against the following methods to demonstrate the robustness of our approach to various forms of uncertainty. We introduce a mix of baselines that correspond to end-to-end, planning, and replanning approaches, with more details in Section C.2.

- **Oracle**: a hand-coded policy that serves as the expert and upper bound for the experiments
- **SmolVLA**: the SmolVLA open-source 0.45B parameter VLA model [117] which was shown to outperform state-of-the-art VLAs with billions of parameters.
- **SmolVLA^{MW}**: SmolVLA fine-tuned on 2,500 episodes of the original MetaWorld benchmark suite
- **PureNN**: a GNN policy operating over LL observations with no access to HL facts and actions
- **PddINN**: a GNN policy operating over LL observations and HL facts but no access to HL actions
- **DetPlan**: the GNN policy from Section 4.2 with a planner [50] in place of an HL policy
- **NdtPlan**: the GNN policy from Section 4.2 with a nondeterministic planner [96] for generating a problem-specific HL policy
- **DetReplan**: same as DetPlan but replan (recompute a new plan) if no HL action is returned due to uncertainty [100, 146, 83, 38, 23, 24]
- **NdtReplan**: same as NdtPlan but replan (compute a new policy) if no HL action is returned
- **BISON (new)**: the bilevel policy for bilevel planning approach using the problem-agnostic HL policy from Section 4.1 and the GNN LL policy from Section 4.2

(Q1) How do VLAs perform on the benchmarks? From Table 2 we observe that pretrained VLA models perform poorly on the benchmarks and fail to solve any task. We observed that this is consistent with existing evaluations of VLAs for the original MetaWorld benchmarks [117], namely how SmolVLA and other VLA approaches achieve at most 45% success rate on *hard* MetaWorld problems [112], which includes the $n = 1$ case of Blocks. Furthermore, we observed that SmolVLA^{MW} can only solve Blocks for $n = 1$ when the block and goal locations are in the same area as in the training set, but not outside. However, we note that VLAs are operating on image and text inputs of the environment, while all other methods assumed processed representations of the inputs.

(Q2) Does BISON generalise to longer horizons than seen in training? From Figure 4, we observe that BISON overall is able to generalise to environments beyond the number of objects that it is trained on, and often generalises better than the end-to-end GNN baselines PureNN and PddINN whose performance often drops on environments beyond 6 objects. This can be attributed to the expressivity limits of GNNs [95, 141] that prohibit them from extrapolating to longer-horizon HL

Table 2: Average success rate ($\uparrow$) and standard deviation of various methods across number of objects and environments. The top 2 scores per column are highlighted.

Type	Method	Blocks ^S	Blocks ^N	Factory ^S	Factory ^N	Colour ^S	Colour ^N	Gacha ^S	Gacha ^N	Total
VLA	SmolVLA	0.00 $\pm$ 0.0	0.00 $\pm$ 0.0	0.00 $\pm$ 0.0	0.00 $\pm$ 0.0	0.00 $\pm$ 0.0	0.00 $\pm$ 0.0	0.00 $\pm$ 0.0	0.00 $\pm$ 0.0	0.00 $\pm$ 0.0
	SmolVLA ^{MW}	0.00 $\pm$ 0.0	0.00 $\pm$ 0.0	0.00 $\pm$ 0.0	0.00 $\pm$ 0.0	0.00 $\pm$ 0.0	0.00 $\pm$ 0.0	0.00 $\pm$ 0.0	0.00 $\pm$ 0.0	0.00 $\pm$ 0.0
GNN	PureNN	0.00 $\pm$ 0.0	0.00 $\pm$ 0.0	0.00 $\pm$ 0.0	0.00 $\pm$ 0.0	0.00 $\pm$ 0.0	0.00 $\pm$ 0.0	0.00 $\pm$ 0.0	0.00 $\pm$ 0.0	0.00 $\pm$ 0.0
	PddlINN	0.33 $\pm$ 0.4	0.20 $\pm$ 0.3	0.18 $\pm$ 0.3	0.07 $\pm$ 0.2	0.12 $\pm$ 0.2	0.36 $\pm$ 0.3	0.51 $\pm$ 0.3	0.60 $\pm$ 0.4	0.30 $\pm$ 0.2
Planning	DetPlan	0.90 $\pm$ 0.1	0.34 $\pm$ 0.3	0.00 $\pm$ 0.0	0.00 $\pm$ 0.0	0.00 $\pm$ 0.0	0.00 $\pm$ 0.0	0.00 $\pm$ 0.0	0.00 $\pm$ 0.0	0.15 $\pm$ 0.3
	NdtPlan	0.99$\pm$0.0	0.39 $\pm$ 0.4	0.00 $\pm$ 0.0	0.00 $\pm$ 0.0	0.61$\pm$0.2	0.01 $\pm$ 0.0	0.00 $\pm$ 0.0	0.00 $\pm$ 0.0	0.25 $\pm$ 0.4
	DetReplan	0.98 $\pm$ 0.0	0.97$\pm$0.1	0.94 $\pm$ 0.1	0.59 $\pm$ 0.3	0.00 $\pm$ 0.0	0.00 $\pm$ 0.0	0.00 $\pm$ 0.0	0.00 $\pm$ 0.0	0.44 $\pm$ 0.4
	NdtReplan	0.99$\pm$0.0	0.97$\pm$0.0	0.94 $\pm$ 0.1	0.51 $\pm$ 0.3	0.26 $\pm$ 0.3	0.17 $\pm$ 0.3	0.00 $\pm$ 0.0	0.00 $\pm$ 0.0	0.48 $\pm$ 0.4
(new)	BISON	0.99$\pm$0.0	0.95 $\pm$ 0.1	0.97$\pm$0.0	0.62$\pm$0.3	0.54 $\pm$ 0.2	0.99$\pm$0.0	0.65$\pm$0.3	0.64$\pm$0.3	0.79$\pm$0.2

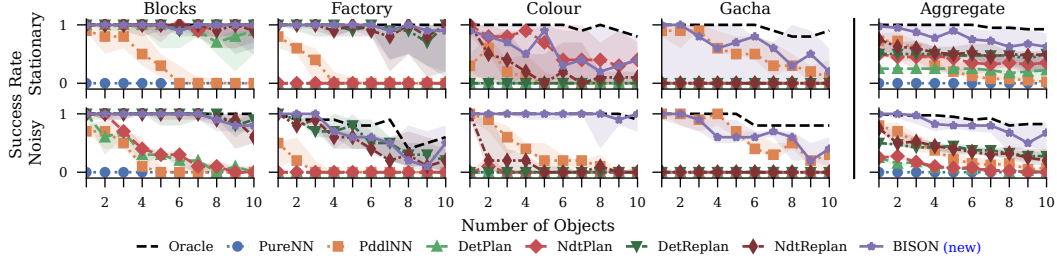


Figure 4: Median (line) and range (shaded) of success rate ($\uparrow$) of methods across number of objects and environments. Results for VLA baselines are omitted as they do not complete any tasks.

planning problems as empirically and theoretically shown in previous works [125, 118, 120]. The best performing seeds often match the upper bound performance of the oracle. From Table 2, BISON matches or outperforms all other baselines with two exceptions. Failures often occur in the covariate shift in the LL execution of HL actions, which suggests that further improvements can be gained from additional post-training, e.g., via RL or DAGger [106].

(Q3) Is BISON robust to uncertainty and open-world environments? As we have seen in the previous question and in Table 2, BISON generally performs best overall, with the exception of two environments where variance in LL policy execution causes failures. Next, we observe that the next best performing approach is NdtReplan which computes a new HL policy π^{hl} per environment episode to be used in the bilevel policy execution framework, and recomputes π^{hl} if it fails to return an action. However, we observe that even with replanning, NdtReplan is less robust to the more complex Colour environments, and furthermore cannot support the open-world Gacha environment.

(Q4) Is BISON more efficient than (re)planning and end-to-end approaches? From Figure 5, we observe that the end-to-end PureNN and PddlINN approaches take longer to train and execute as they encode more information in the LL GNN policy architecture than the one from Section 4.2: PureNN encodes all LL information and PddlINN encodes all LL and HL information, while BISON’s LL policy only encodes the HL action and corresponding object information. Single-shot planners (DetPlan, NdtPlan) are comparable to BISON as planning costs are amortised over the episode, but achieve poor performance. Replanning methods (DetReplan, NdtReplan) take at least as long as BISON due to replanning upon encountering unexpected uncertainty.

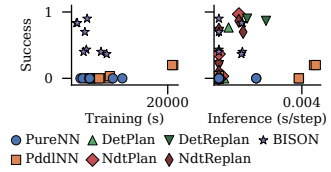


Figure 5: Success rate ($\uparrow$) vs. time ($\downarrow$) for training and inference with 10 objects.

(Q5) Do learned HL policies generalise to arbitrary numbers of objects? We answer this question by inspecting the learned policies and evaluating their performance on HL planning problems over numbers of objects. Learned HL policies in BISON are symbolic and hence can be interpreted manually or by an LLM to generalise over arbitrary numbers of objects, as displayed in Section C.4. We also perform experiments on corresponding PDDL Blocks^S planning problems ranging from 3 to 10,000 blocks. We compare BISON’s solving time to a state-of-the-art

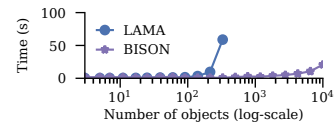


Figure 6: HL solving time ($\downarrow$) vs. number of objects.

PDDL planner, LAMA [105], with a 100s timeout. We observe from Figure 6 that BISON solves problems with 10,000 blocks in a few seconds while LAMA struggles with a few hundred objects.

6 Discussion, Limitations, and Conclusion

We introduced BISON, a system to learn bilevel policies ($\pi^{\text{hl}}, \pi^{\text{ll}}$) from LL demonstrations paired with HL goals that generalise to long-horizon problems. We learn π^{hl} policies from abstracted symbolic traces of the LL demonstrations using goal regression and inductive generalisation to produce first-order, condition-action rules that can generalise to arbitrary numbers of objects, and realise our π^{ll} policies via graph neural networks with fewer than 33k parameters for executing HL actions returned by π^{hl} in the LL action space. The novelty of our approach is at least two-fold: (1) in contrast to existing bilevel planning approaches that search for HL plans over symbolic abstractions before refining them in the LL environment, BISON inductively generalises HL policies from LL demonstrations without the need for search, and (2) the encoding of our policies as first-order, condition-action rules is novel in the context of bilevel planning and facilitates open-world planning and generalisation. Through this novel insight and approach, we can learn compact policies that demonstrate strong generalisation capabilities and efficiency compared to state-of-the-art VLA, end-to-end, and planning baselines on extended MetaWorld benchmarks.

Limitations Our approach inherits the limitations of bilevel planning, in particular, the assumption of a learned or given HL abstraction that provides useful guidance for LL planning (there is much research on how to find good abstractions as discussed in Section 3). Another limitation of our approach is that we have no guarantees of optimality and generalisation of the LL policies.

Acknowledgements

We gratefully acknowledge funding from the Natural Sciences and Engineering Research Council of Canada (NSERC) and the Canada CIFAR AI Chairs Program. Till Hofmann was funded by the Federal Ministry of Education and Research (BMBF) and the Ministry of Culture and Science of the German State of North Rhine-Westphalia (MKW) under the Excellence Strategy of the Federal Government and the Länder. Resources used in preparing this research were provided, in part, by the Province of Ontario, the Government of Canada through CIFAR, and companies sponsoring the Vector Institute. We thank Frieda Rong and William Shen for helpful discussions.

References

- [1] David Abel, Dilip Arumugam, Lucas Lehnert, and Michael L. Littman. State abstractions for lifelong reinforcement learning. In *ICML*, 2018.
- [2] David Abel, D. Ellis Hershkowitz, and Michael L. Littman. Near optimal behavior via approximate state abstraction. In *ICML*, 2016.
- [3] Zergham Ahmed, Joshua B. Tenenbaum, Christopher Bates, and Samuel J. Gershman. Synthesizing world models for bilevel planning. *Trans. Mach. Learn. Res.*, 2025, 2025.
- [4] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alexander Herzog, Daniel Ho, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Eric Jang, Rosario Jauregui Ruano, Kyle Jeffrey, Sally Jesmonth, Nikhil J. Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Kuang-Huei Lee, Sergey Levine, Yao Lu, Linda Luu, Carolina Parada, Peter Pastor, Jornell Quiambao, Kanishka Rao, Jarek Rettinghouse, Diego Reyes, Pierre Sermanet, Nicolas Sievers, Clayton Tan, Alexander Toshev, Vincent Vanhoucke, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, and Mengyuan Yan. Do as I can, not as I say: Grounding language in robotic affordances. In *CoRL*, 2022.
- [5] Ankesh Anand, Evan Racah, Sherjil Ozair, Yoshua Bengio, Marc-Alexandre Côté, and R. Devon Hjelm. Unsupervised state representation learning in Atari. In *NeurIPS*, 2019.
- [6] Masataro Asai and Alex Fukunaga. Classical planning in deep latent space: Bridging the subsymbolic-symbolic boundary. In *AAAI*, 2018.

- [7] Masataro Asai, Hiroshi Kajino, Alex Fukunaga, and Christian Muise. Classical planning in deep latent space. *J. Artif. Intell. Res.*, 74:1599–1686, 2022.
- [8] Ashay Athalye, Nishanth Kumar, Tom Silver, Yichao Liang, Jiuguang Wang, Tomás Lozano-Pérez, and Leslie Pack Kaelbling. From pixels to predicates: Learning symbolic world models via pretrained VLMs. *IEEE Robotics and Automation Letters*, 11(4):4002–4009, 2026.
- [9] Fahiem Bacchus and Qiang Yang. Downward refinement and the efficiency of hierarchical problem solving. *Artif. Intell.*, 71(1):43–100, 1994.
- [10] Pascal Bercher, Ron Alford, and Daniel Höller. A survey on hierarchical planning - one abstract idea, many concrete realizations. In *IJCAI*, 2019.
- [11] Luigi Bonassi, Giuseppe De Giacomo, Marco Favorito, Francesco Fuggitti, Alfonso Emilio Gerevini, and Enrico Scala. Planning for temporally extended goals in pure-past linear temporal logic. *Artif. Intell.*, 348:104409, 2025.
- [12] Smail Ait Bouhsain, Rachid Alami, and Thierry Siméon. Learning to predict action feasibility for task and motion planning in 3d environments. In *ICRA*, 2023.
- [13] Smail Ait Bouhsain, Rachid Alami, and Thierry Siméon. Learning geometric reasoning networks for robot task and motion planning. In *ICLR*, 2025.
- [14] Craig Boutilier and Richard Dearden. Using abstractions for decision-theoretic planning with time constraints. In *AAAI*, 1994.
- [15] Alper Kamil Bozkurt, Yu Wang, Michael M. Zavlanos, and Miroslav Pajic. Control synthesis from linear temporal logic specifications using model-free reinforcement learning. In *ICRA*, 2020.
- [16] Christopher P. Burgess, Loïc Matthey, Nicholas Watters, Rishabh Kabra, Irina Higgins, Matthew M. Botvinick, and Alexander Lerchner. MONet: Unsupervised scene decomposition and representation. *CoRR*, abs/1901.11390, 2019.
- [17] Alberto Camacho, Rodrigo Toro Icarte, Toryn Q. Klassen, Richard Anthony Valenzano, and Sheila A. McIlraith. LTL and beyond: Formal languages for reward function specification in reinforcement learning. In *IJCAI*, 2019.
- [18] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *RSS*, 2023.
- [19] Rohan Chitnis, Dylan Hadfield-Menell, Abhishek Gupta, Siddharth Srivastava, Edward Grosse, Christopher Lin, and Pieter Abbeel. Guided search for task and motion plans using learned heuristics. In *ICRA*, 2016.
- [20] Rohan Chitnis, Tom Silver, Joshua B. Tenenbaum, Tomás Lozano-Pérez, and Leslie Pack Kaelbling. Learning neuro-symbolic relational transition models for bilevel planning. In *IROS*, 2022.
- [21] Bartłomiej Cieslar, Leslie Pack Kaelbling, Tomás Lozano-Pérez, and Jorge Mendez-Mendez. Learning long-horizon action dependencies in sampling-based bilevel planning. In *CoRL*, 2024.
- [22] Augusto B. Corrêa, Florian Pommerening, Malte Helmert, and Guillem Francès. Lifted successor generation using query optimization techniques. In *ICAPS*, 2020.
- [23] Aidan Curtis, Xiaolin Fang, Leslie Pack Kaelbling, Tomás Lozano-Pérez, and Caelan Reed Garrett. Long-horizon manipulation of unknown objects via task and motion planning with estimated affordances. In *ICRA*, 2022.
- [24] Aidan Curtis, George Matheos, Nishad Gothoskar, Vikash Mansinghka, Joshua B. Tenenbaum, Tomás Lozano-Pérez, and Leslie Pack Kaelbling. Partially observable task and motion planning with uncertainty and risk awareness. In *RSS*, 2024.

- [25] Aidan Curtis, Tom Silver, Joshua B. Tenenbaum, Tomás Lozano-Pérez, and Leslie Pack Kaelbling. Discovering state and action abstractions for generalized task and motion planning. In *AAAI*, 2022.
- [26] Neil T. Dantam, Zachary K. Kingston, Swarat Chaudhuri, and Lydia E. Kavraki. An incremental constraint-based framework for task and motion planning. *Int. J. Robotics Res.*, 37(10), 2018.
- [27] Peter Dayan and Geoffrey E. Hinton. Feudal reinforcement learning. In *NeurIPS*, 1992.
- [28] Thomas L. Dean and Robert Givan. Model minimization in Markov decision processes. In *AAAI*, 1997.
- [29] Thomas G. Dietterich. Hierarchical reinforcement learning with the MAXQ value function decomposition. *J. Artif. Intell. Res.*, 13:227–303, 2000.
- [30] Danny Driess, Jung-Su Ha, and Marc Toussaint. Deep visual reasoning: Learning to predict action sequences for task and motion planning from an initial scene image. In *RSS*, 2020.
- [31] Danny Driess, Ozgur S. Oguz, Jung-Su Ha, and Marc Toussaint. Deep visual heuristics: Learning feasibility of mixed-integer programs for manipulation planning. In *ICRA*, 2020.
- [32] Danny Driess, Fei Xia, Mehdi S. M. Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, Wenlong Huang, Yevgen Chebotar, Pierre Sermanet, Daniel Duckworth, Sergey Levine, Vincent Vanhoucke, Karol Hausman, Marc Toussaint, Klaus Greff, Andy Zeng, Igor Mordatch, and Pete Florence. Palm-e: An embodied multimodal language model. In *ICML*, 2023.
- [33] Qiwei Du, Bowen Li, Yi Du, Shaoshu Su, Taimeng Fu, Zitong Zhan, Zhipeng Zhao, and Chen Wang. Fast task planning with neuro-symbolic relaxation. *IEEE Robotics Autom. Lett.*, 11(3):3684–3691, 2026.
- [34] Nouha Dziri, Ximing Lu, Melanie Sclar, Xiang Lorraine Li, Liwei Jiang, Bill Yuchen Lin, Sean Welleck, Peter West, Chandra Bhagavatula, Ronan Le Bras, Jena D. Hwang, Soumya Sanyal, Xiang Ren, Allyson Ettinger, Zaïd Harchaoui, and Yejin Choi. Faith and fate: Limits of transformers on compositionality. In *NeurIPS*, 2023.
- [35] Kutluhan Erol, James A. Hendler, and Dana S. Nau. Complexity results for HTN planning. *Ann. Math. Artif. Intell.*, 18(1):69–93, 1996.
- [36] Richard Fikes, Peter E. Hart, and Nils J. Nilsson. Learning and executing generalized robot plans. *Artif. Intell.*, 3(1-3):251–288, 1972.
- [37] Richard Fikes and Nils J. Nilsson. STRIPS: A New Approach to the Application of Theorem Proving to Problem Solving. *Artif. Intell.*, 2(3/4):189–208, 1971.
- [38] Christian Fritz and Sheila A. McIlraith. Monitoring plan optimality during execution. In *ICAPS*, 2007.
- [39] Caelan Reed Garrett, Rohan Chitnis, Rachel M. Holladay, Beomjoon Kim, Tom Silver, Leslie Pack Kaelbling, and Tomás Lozano-Pérez. Integrated task and motion planning. *Annu. Rev. Control. Robotics Auton. Syst.*, 4:265–293, 2021.
- [40] Hector Geffner and Blai Bonet. *A Concise Introduction to Models and Methods for Automated Planning*. Synthesis Lectures on Artificial Intelligence and Machine Learning. Morgan & Claypool Publishers, 2013.
- [41] Malik Ghallab, Dana S. Nau, and Paolo Traverso. *Automated planning - theory and practice*. Elsevier, 2004.
- [42] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *ICML*, 2017.
- [43] Fausto Giunchiglia and Toby Walsh. A theory of abstraction. *Artif. Intell.*, 57:323–389, 1992.

- [44] C. Cordell Green. Application of theorem proving to problem solving. In Donald E. Walker and Lewis M. Norton, editors, *IJCAI*, pages 219–240. William Kaufmann, 1969.
- [45] Charles Gretton and Sylvie Thiébaux. Exploiting first-order regression in inductive policy selection. In *UAI*, 2004.
- [46] Muzhi Han, Yifeng Zhu, Song-Chun Zhu, Ying Nian Wu, and Yuke Zhu. INTERPRET: interactive predicate learning from language feedback for generalizable task planning. In *RSS*, 2024.
- [47] Dan Haramati, Tal Daniel, and Aviv Tamar. Entity-centric reinforcement learning for object manipulation from pixels. In *ICLR*, 2024.
- [48] Patrik Haslum, Nir Lipovetzky, Daniele Magazzeni, and Christian Muise. *An Introduction to the Planning Domain Definition Language*. 2019.
- [49] Milos Hauskrecht, Nicolas Meuleau, Leslie Pack Kaelbling, Thomas L. Dean, and Craig Boutilier. Hierarchical solution of Markov decision processes using macro-actions. In *UAI*, 1998.
- [50] Malte Helmert. The fast downward planning system. *J. Artif. Intell. Res.*, 26:191–246, 2006.
- [51] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *NeurIPS*, 2020.
- [52] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, DDL Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. In *NeurIPS*, 2022.
- [53] Yingdong Hu, Fanqi Lin, Tong Zhang, Li Yi, and Yang Gao. Look before you leap: Unveiling the power of GPT-4V in robotic vision-language planning. *CoRR*, abs/2311.17842, 2023.
- [54] Jinbang Huang, Allen Tao, Rozilyn Marco, Miroslav Bogdanovic, Jonathan Kelly, and Florian Shkurti. Automated planning domain inference for task and motion planning. In *ICRA*, 2025.
- [55] Wenlong Huang, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, Jonathan Tompson, Igor Mordatch, Yevgen Chebotar, Pierre Sermanet, Tomas Jackson, Noah Brown, Linda Luu, Sergey Levine, Karol Hausman, and Brian Ichter. Inner monologue: Embodied reasoning through planning with language models. In *CoRL*, 2022.
- [56] León Illanes, Xi Yan, Rodrigo Toro Icarte, and Sheila A. McIlraith. Symbolic plans as high-level instructions for reinforcement learning. In *ICAPS*, 2020.
- [57] Physical Intelligence, Bo Ai, Ali Amin, Raichelle Aniceto, Ashwin Balakrishna, Greg Balke, Kevin Black, George Bokinsky, Shihao Cao, Thomas Charbonnier, Vedant Choudhary, Foster Collins, Ken Conley, Grace Connors, James Darpinian, Karan Dhabalia, Maitrayee Dhaka, Jared DiCarlo, Danny Driess, Michael Equi, Adnan Esmail, Yunhao Fang, Chelsea Finn, Catherine Glossop, Thomas Godden, Ivan Goryachev, Lachlan Groom, Haroun Habeeb, Hunter Hancock, Karol Hausman, Gashon Hussein, Victor Hwang, Brian Ichter, Connor Jacobsen, Szymon Jakubczak, Rowan Jen, Tim Jones, Gregg Kammerer, Ben Katz, Liyiming Ke, Mairbek Khadikov, Chandra Kuchi, Marinda Lamb, Devin LeBlanc, Brendon LeCount, Sergey Levine, Xinyu Li, Adrian Li-Bell, Vladislav Lialin, Zhonglin Liang, Wallace Lim, Yao Lu, Enyu Luo, Vishnu Mano, Nandan Marwaha, Aikys Mongush, Liam Murphy, Suraj Nair, Tyler Patterson, Karl Pertsch, Allen Z. Ren, Gavin Schelske, Charvi Sharma, Baifeng Shi, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, Will Stoeckle, Jiaming Tang, Jimmy Tanner, Shalom Tekeste, Marcel Torne, Kyle Vedder, Quan Vuong, Anna Walling, Haohuan Wang, Jason Wang, XuDong Wang, Chris Whalen, Samuel Whitmore, Blake Williams, Charles Xu, Sukwon Yoo, Lili Yu, Wuming Zhang, Zhuoyang Zhang, and Ury Zhilinsky. $\pi_{0.7}$: a steerable generalist robotic foundation model with emergent capabilities. *CoRR*, abs/2604.15483, 2026.

- [58] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Manuel Y. Galliker, Dibya Ghosh, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Devin LeBlanc, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Allen Z. Ren, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, James Tanner, Quan Vuong, Homer Walke, Anna Walling, Haohuan Wang, Lili Yu, and Ury Zhilinsky. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *CoRR*, abs/2504.16054, 2025.
- [59] Mathias Jackermeier and Alessandro Abate. Deepltl: Learning to efficiently satisfy complex LTL specifications for multi-task RL. In *ICLR*, 2025.
- [60] Steven James, Benjamin Rosman, and George Konidaris. Autonomous learning of object-centric abstractions for high-level planning. In *ICLR*, 2022.
- [61] Subbarao Kambhampati, Karthik Valmeekam, Lin Guan, Mudit Verma, Kaya Stechly, Siddhant Bhambri, Lucas Saldyt, and Anil Murthy. Position: Llms can’t plan, but can help planning in llm-modulo frameworks. In *ICML*, 2024.
- [62] Mohamed Khodeir, Ben Agro, and Florian Shkurti. Learning to search in task and motion planning with streams. *IEEE Robotics Autom. Lett.*, 8(4):1983–1990, 2023.
- [63] Beomjoon Kim, Zi Wang, Leslie Pack Kaelbling, and Tomás Lozano-Pérez. Learning to guide task and motion planning using score-space representation. *Int. J. Robotics Res.*, 38(7), 2019.
- [64] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Paul Foster, Pannag R. Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. Openvla: An open-source vision-language-action model. In *CoRL*, 2024.
- [65] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *ICLR*, 2015.
- [66] Thomas N. Kipf, Elise van der Pol, and Max Welling. Contrastive learning of structured world models. In *ICLR*, 2020.
- [67] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *ICLR*, 2017.
- [68] George Konidaris. On the necessity of abstraction. *Current Opinion in Behavioral Sciences*, 29:1–7, 2019. Artificial Intelligence.
- [69] George Dimitri Konidaris and Andrew G. Barto. Efficient skill learning using abstraction selection. In *IJCAI*, 2009.
- [70] George Dimitri Konidaris, Leslie Pack Kaelbling, and Tomás Lozano-Pérez. Constructing symbolic representations for high-level planning. In *AAAI*, 2014.
- [71] George Dimitri Konidaris, Leslie Pack Kaelbling, and Tomás Lozano-Pérez. From skills to symbols: Learning symbolic representations for abstract high-level planning. *J. Artif. Intell. Res.*, 61:215–289, 2018.
- [72] Richard E. Korf. Planning as search: A quantitative approach. *Artif. Intell.*, 33(1):65–88, 1987.
- [73] Nishanth Kumar, Tom Silver, Willie McClinton, Linfeng Zhao, Stephen Proulx, Tomás Lozano-Pérez, Leslie Pack Kaelbling, and Jennifer L. Barry. Practice makes perfect: Planning to learning skill parameter policies. In *RSS*, 2024.
- [74] Hoang Minh Le, Nan Jiang, Alekh Agarwal, Miroslav Dudík, Yisong Yue, and Hal Daumé III. Hierarchical imitation and reinforcement learning. In *ICML*, 2018.
- [75] Andrew C. Li, Toryn Q. Klassen, Andrew Wang, Parand A. Alamdari, and Sheila A. McIlraith. Ground-compose-reinforce: Grounding language inagentic behaviours using limited data. In *NeurIPS*, 2025.

- [76] Bowen Li, Tom Silver, Sebastian A. Scherer, and Alexander G. Gray. Bilevel learning for bilevel planning. In *RSS*, 2025.
- [77] Lihong Li, Thomas J. Walsh, and Michael L. Littman. Towards a unified theory of state abstraction for mdps. In *International Symposium on Artificial Intelligence and Mathematics*, 2006.
- [78] Xiao Li, Cristian Ioan Vasile, and Calin Belta. Reinforcement learning with temporal logic rewards. In *IROS*, 2017.
- [79] Yichao Liang, Nishanth Kumar, Hao Tang, Adrian Weller, Joshua B. Tenenbaum, Tom Silver, João F. Henriques, and Kevin Ellis. Visualpredicator: Learning abstract world models with neuro-symbolic predicates for robot planning. In *ICLR*, 2025.
- [80] Yichao Liang, Dat Nguyen, Cambridge Yang, Tianyang Li, Joshua B. Tenenbaum, Carl Edward Rasmussen, Adrian Weller, Zenna Tavares, Tom Silver, and Kevin Ellis. Exopredicator: Learning abstract models of dynamic worlds for robot planning. In *ICLR*, 2026.
- [81] Bill Yuchen Lin, Ronan Le Bras, Kyle Richardson, Ashish Sabharwal, Radha Poovendran, Peter Clark, and Yejin Choi. Zebralogic: On the scaling limits of llms for logical reasoning. In *ICML*, 2025.
- [82] Yixin Lin, Austin S. Wang, Eric Undersander, and Akshara Rai. Efficient and interpretable robot manipulation with graph neural networks. *IEEE Robotics Autom. Lett.*, 7(2):2740–2747, 2022.
- [83] Iain Little, Sylvie Thiebaux, et al. Probabilistic planning vs. replanning. In *ICAPS*, 2007.
- [84] Bo Liu, Yuqian Jiang, Xiaohan Zhang, Qiang Liu, Shiqi Zhang, Joydeep Biswas, and Peter Stone. LLM+P: empowering large language models with optimal planning proficiency. *CoRR*, abs/2304.11477, 2023.
- [85] Xiaotian Liu, Ali Pesaranghader, Hanze Li, Punyaphat Sukcharoenchaikul, Jaehong Kim, Tanmana Sadhu, Hyejeong Jeon, and Scott Sanner. Open-world planning via lifted regression with llm-inferred affordances for embodied agents. In *ACL*, 2025.
- [86] Ilya Loshchilov and Frank Hutter. SGDR: stochastic gradient descent with warm restarts. In *ICLR*, 2017.
- [87] Tomás Lozano-Pérez and Leslie Pack Kaelbling. A constraint-based method for solving sequential manipulation planning problems. In *IROS*, 2014.
- [88] Tomas Lozano-Perez, Matthew T Mason, and Russell H Taylor. Automatic synthesis of fine-motion strategies for robots. *Int. J. Robot. Res.*, 3(1):3–24, 1984.
- [89] Ajay Mandlekar, Caelan Reed Garrett, Danfei Xu, and Dieter Fox. Human-in-the-loop task and motion planning for imitation learning. In *CoRL*, 2023.
- [90] Bhaskara Marthi, Stuart Russell, and Jason Andrew Wolfe. Angelic hierarchical planning: Optimal and online algorithms. In *ICAPS*, 2008.
- [91] Drew McDermott, Malik Ghallab, Adele E. Howe, Craig A. Knoblock, Ashwin Ram, Manuela M. Veloso, Daniel S. Weld, and David E. Wilkins. PDDL-the Planning Domain Definition Language. Technical report, 1998.
- [92] Michael James McDonald and Dylan Hadfield-Menell. Guided imitation of task and motion planning. In *CoRL*, 2021.
- [93] Reginald McLean, Evangelos Chatzaroulas, Luc McCutcheon, Frank Röder, Tianhe Yu, Zhanpeng He, K. R. Zentner, Ryan Julian, J K Terry, Isaac Woungang, Nariman Farsad, and Pablo Samuel Castro. Meta-world+: An improved, standardized, RL benchmark. In *NeurIPS*, 2025.
- [94] Jorge Mendez-Mendez, Leslie Pack Kaelbling, and Tomás Lozano-Pérez. Embodied lifelong learning for task and motion planning. In *CoRL*, 2023.

- [95] Christopher Morris, Martin Ritzert, Matthias Fey, William L. Hamilton, Jan Eric Lenssen, Gaurav Rattan, and Martin Grohe. Weisfeiler and leman go neural: Higher-order graph neural networks. In *AAAI*, 2019.
- [96] Christian Muise, Sheila A. McIlraith, and J. Christopher Beck. PRP rebooted: Advancing the state of the art in FOND planning. In *AAAI*, 2024.
- [97] Dana S. Nau, Tsz-Chiu Au, Okhtay Ilghami, Ugur Kuter, J. William Murdock, Dan Wu, and Fusun Yaman. SHOP2: an HTN planning system. *J. Artif. Intell. Res.*, 20:379–404, 2003.
- [98] Seohong Park, Kevin Frans, Deepinder Mann, Benjamin Eysenbach, Aviral Kumar, and Sergey Levine. Horizon reduction makes rl scalable. In *NeurIPS*, 2025.
- [99] Sung-Gil Park, Yong-Geon Kim, Seuk-Woo Ryu, Byeong Gil Yoo, Sungeun Chung, Jeong-Seop Park, Woo-Jin Ahn, and Myo-Taeg Lim. Robust vision-language-action models via object-centric learning and distance-based chunk alignment. *Applied Sciences*, 16(7), 2026.
- [100] Ola Pettersson. Execution monitoring in robotics: A survey. *Robotics and Autonomous Systems*, 53(2):73–88, 2005.
- [101] Wenjie Qiu, Wensen Mao, and He Zhu. Instructing goal-conditioned reinforcement learning agents with temporal logic objectives. In *NeurIPS*, 2023.
- [102] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *ICML*, 2021.
- [103] Raymond Reiter. The frame problem in the situation calculus: A simple solution (sometimes) and a completeness result for goal regression. In *Artificial and Mathematical Theory of Computation*. Academic Press, 1991.
- [104] Raymond Reiter. *Knowledge in Action: Logical Foundations for Specifying and Implementing Dynamical Systems*. MIT Press, 2001.
- [105] Silvia Richter and Matthias Westphal. The LAMA planner: Guiding cost-based anytime planning with landmarks. *J. Artif. Intell. Res.*, 39:127–177, 2010.
- [106] Stéphane Ross, Geoffrey J. Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *AISTATS*, 2011.
- [107] Earl D. Sacerdoti. Planning in a hierarchy of abstraction spaces. *Artif. Intell.*, 5(2):115–135, 1974.
- [108] Scott Sanner and Craig Boutilier. Practical solution techniques for first-order mdps. *Artif. Intell.*, 173(5-6):748–788, 2009.
- [109] Enrico Scala, Patrik Haslum, Sylvie Thiébaux, and Miquel Ramírez. Subgoal techniques for satisficing and optimal numeric planning. *J. Artif. Intell. Res.*, 68:691–752, 2020.
- [110] Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini. The graph neural network model. *IEEE Trans. Neural Networks*, 20(1):61–80, 2009.
- [111] Jendrik Seipp, Thomas Keller, and Malte Helmert. Saturated cost partitioning for optimal classical planning. *J. Artif. Intell. Res.*, 67:129–167, 2020.
- [112] Younggyo Seo, Danijar Hafner, Hao Liu, Fangchen Liu, Stephen James, Kimin Lee, and Pieter Abbeel. Masked world models for visual control. In *CoRL*, 2022.
- [113] Naman Shah, Jayesh Nagpal, Pulkit Verma, and Siddharth Srivastava. From real world to logic and back: Learning generalizable relational concepts for long horizon robot planning. In *CoRL*, 2025.

- [114] Naman Shah, Deepak Kala Vasudevan, Kislay Kumar, Pranav Kamojjhal, and Siddharth Srivastava. Anytime integrated task and motion policies for stochastic environments. In *ICRA*, 2020.
- [115] William Shen, Caelan Reed Garrett, Ankit Goyal, Tucker Hermans, and Fabio Ramos. Differentiable gpu-parallelized task and motion planning. In *RSS*, 2025.
- [116] William Shen, Nishanth Kumar, Sahit Chintalapudi, Jie Wang, Christopher Watson, Edward Hu, Jing Cao, Dinesh Jayaraman, Leslie Pack Kaelbling, and Tomás Lozano-Pérez. Tiptop: A modular open-vocabulary planning system for robotic manipulation. *CoRR*, abs/2603.09971, 2026.
- [117] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andrés Marafioti, Simon Alibert, Matthieu Cord, Thomas Wolf, and Rémi Cadène. Smolvla: A vision-language-action model for affordable and efficient robotics. *CoRR*, abs/2506.01844, 2025.
- [118] Tom Silver, Ashay Athalye, Joshua B. Tenenbaum, Tomás Lozano-Pérez, and Leslie Pack Kaelbling. Learning neuro-symbolic skills for bilevel planning. In *CoRL*, 2022.
- [119] Tom Silver, Rohan Chitnis, Aidan Curtis, Joshua B. Tenenbaum, Tomás Lozano-Pérez, and Leslie Pack Kaelbling. Planning with learned object importance in large problem instances using graph neural networks. In *AAAI*, 2021.
- [120] Tom Silver, Rohan Chitnis, Nishanth Kumar, Willie McClinton, Tomás Lozano-Pérez, Leslie Pack Kaelbling, and Joshua B. Tenenbaum. Predicate invention for bilevel planning. In *AAAI*, 2023.
- [121] Tom Silver, Rohan Chitnis, Joshua B. Tenenbaum, Leslie Pack Kaelbling, and Tomás Lozano-Pérez. Learning symbolic operators for task and motion planning. In *IROS*, 2021.
- [122] Herbert A Simon. Rational choice and the structure of the environment. *Psychol. Rev.*, 63(2):129, 1956.
- [123] David Speck, Jendrik Seipp, and Álvaro Torralba. Symbolic search for cost-optimal planning with expressive model extensions. *J. Artif. Intell. Res.*, 82, 2025.
- [124] Siddharth Srivastava, Eugene Fang, Lorenzo Riano, Rohan Chitnis, Stuart Russell, and Pieter Abbeel. Combined task and motion planning through an extensible planner-independent interface layer. In *ICRA*, 2014.
- [125] Simon Ståhlberg, Blai Bonet, and Hector Geffner. Learning general optimal policies with graph neural networks: Expressive power, transparency, and limits. In *ICAPS*, 2022.
- [126] Richard S. Sutton, Doina Precup, and Satinder Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artif. Intell.*, 112(1-2):181–211, 1999.
- [127] Austin Tate. Generating project networks. In *IJCAI*, 1977.
- [128] Gemini Team. Gemini: A family of highly capable multimodal models. *CoRR*, 2023.
- [129] SIMA Team and Google DeepMind. SIMA 2: A generalist embodied agent for virtual worlds. *CoRR*, abs/2512.04797, 2025.
- [130] Rodrigo Toro Icarte, Toryn Q. Klassen, Richard Anthony Valenzano, and Sheila A. McIlraith. Teaching multiple tasks to an RL agent using LTL. In *AAMAS*, 2018.
- [131] Rodrigo Toro Icarte, Toryn Q. Klassen, Richard Anthony Valenzano, and Sheila A. McIlraith. Reward machines: Exploiting reward function structure in reinforcement learning. *J. Artif. Intell. Res.*, 73:173–208, 2022.
- [132] Marc Toussaint. Logic-geometric programming: An optimization-based approach to combined task and motion planning. In *IJCAI*, 2015.

- [133] Pashootan Vaezipoor, Andrew C. Li, Rodrigo Toro Icarte, and Sheila A. McIlraith. LTL2action: Generalizing LTL instructions for multi-task RL. In *ICML*, 2021.
- [134] Cameron Voloshin, Hoang Minh Le, Swarat Chaudhuri, and Yisong Yue. Policy optimization with linear temporal logic constraints. In *NeurIPS*, 2022.
- [135] Richard J. Waldinger. Achieving several goals simultaneously. *Machine Intelligence*, 8:94–136, 1977.
- [136] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, Ed H. Chi, Tatsunori Hashimoto, Oriol Vinyals, Percy Liang, Jeff Dean, and William Fedus. Emergent abilities of large language models. *Trans. Mach. Learn. Res.*, 2022, 2022.
- [137] Andrew M. Wells, Neil T. Dantam, Anshumali Shrivastava, and Lydia E. Kavraki. Learning feasibility for task and motion planning in tabletop environments. *IEEE Robotics Autom. Lett.*, 4(2):1255–1262, 2019.
- [138] Bowen Wen, Matthew Trepte, Joseph Aribido, Jan Kautz, Orazio Gallo, and Stan Birchfield. Foundationstereo: Zero-shot stereo matching. In *CVPR*, 2025.
- [139] Kai Xi, Stephen Gould, and Sylvie Thiébaux. Neuro-symbolic learning of lifted action models from visual traces. In *ICAPS*, 2024.
- [140] Danfei Xu, Roberto Martín-Martín, De-An Huang, Yuke Zhu, Silvio Savarese, and Li Fei-Fei. Regression planning networks. In *NeurIPS*, 2019.
- [141] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *ICLR*, 2019.
- [142] Lei Xu, Tianyu Ren, Georgia Chalvatzaki, and Jan Peters. Accelerating integrated task and motion planning with neural feasibility checking. *CoRR*, abs/2203.10568, 2022.
- [143] Beyazit Yalcinkaya, Niklas Lauffer, Marcell Vazquez-Chanlatte, and Sanjit A. Seshia. Compositional automata embeddings for goal-conditioned reinforcement learning. In *NeurIPS*, 2024.
- [144] Zhutian Yang, Caelan Reed Garrett, Tomás Lozano-Pérez, Leslie Pack Kaelbling, and Dieter Fox. Sequence-based plan feasibility prediction for efficient task and motion planning. In *RSS*, 2023.
- [145] Ziyi Yang, Benned Hedegaard, Ahmed Jaafar, Yichen Wei, Skye Thompson, Shreyas Sundara Raman, Haotian Fu, Stefanie Tellex, George Dimitri Konidaris, David Paulius, and Naman Shah. Skillwrapper: Generative predicate invention for skill abstraction. *CoRR*, abs/2511.18203, 2025.
- [146] Sung Wook Yoon, Alan Fern, and Robert Givan. FF-Replan: A baseline for probabilistic planning. In *ICAPS*, 2007.
- [147] Tianhe Yu, Deirdre Quillen, Zhanpeng He, Ryan Julian, Karol Hausman, Chelsea Finn, and Sergey Levine. Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning. In *CoRL*, 2019.
- [148] Wentao Yuan, Adithyavairavan Murali, Arsalan Mousavian, and Dieter Fox. M2T2: multi-task masked transformer for object-centric pick and place. In *CoRL*, 2023.
- [149] Andrii Zadaianchuk, Maximilian Seitzer, and Georg Martius. Self-supervised visual reinforcement learning with object-centric representations. In *ICLR*, 2021.
- [150] Wancong Zhang, Basile Terver, Artem Zholus, Soham Chitnis, Harsh Sutaria, Mido Assran, Randall Balestriero, Amir Bar, Adrien Bardes, Yann LeCun, and Nicolas Ballas. Hierarchical planning with latent world models. *CoRR*, abs/2604.03208, 2026.

- [151] Zhigen Zhao, Shuo Cheng, Yan Ding, Ziyi Zhou, Shiqi Zhang, Danfei Xu, and Ye Zhao. A survey of optimization-based task and motion planning: From classical to learning approaches. *IEEE/ASME Transactions on Mechatronics*, 30(4):2799–2825, 2025.
- [152] Yifeng Zhu, Jonathan Tremblay, Stan Birchfield, and Yuke Zhu. Hierarchical planning for long-horizon manipulation with geometric and symbolic scene graphs. In *ICRA*, 2021.
- [153] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, Quan Vuong, Vincent Vanhoucke, Huong T. Tran, Radu Soricut, Anikait Singh, Jaspiar Singh, Pierre Sermanet, Pannag R. Sanketi, Grecia Salazar, Michael S. Ryoo, Krista Reymann, Kanishka Rao, Karl Pertsch, Igor Mordatch, Henryk Michalewski, Yao Lu, Sergey Levine, Lisa Lee, Tsang-Wei Edward Lee, Isabel Leal, Yuheng Kuang, Dmitry Kalashnikov, Ryan Julian, Nikhil J. Joshi, Alex Irpan, Brian Ichter, Jasmine Hsu, Alexander Herzog, Karol Hausman, Keerthana Gopalakrishnan, Chuyuan Fu, Pete Florence, Chelsea Finn, Kumar Avinava Dubey, Danny Driess, Tianli Ding, Krzysztof Marcin Choromanski, Xi Chen, Yevgen Chebotar, Justice Carbajal, Noah Brown, Anthony Brohan, Montserrat Gonzalez Arenas, and Kehang Han. RT-2: vision-language-action models transfer web knowledge to robotic control. In *CoRL*, 2023.

Appendix

Contents

1	Introduction	2
2	Related Work	2
3	Problem Statement	3
4	Learning Bilevel Policies for Bilevel Planning	4
4.1	Learning HL Policies via Goal Regression and Inductive Generalisation	5
4.2	Learning LL Policies via Graph Neural Networks and Imitation Learning	7
5	Experiments	8
6	Discussion, Limitations, and Conclusion	10
A	Summary of Definitions and Notation	21
B	Theory	21
B.1	Goal Independence	21
B.2	Object-Renaming Equivalence of HL Problems	22
B.3	Generalisation Result	22
C	Additional Experimental Details	23
C.1	GNN Hyperparameters	23
C.2	Baselines	23
C.3	Benchmarks	24
C.4	Visualising Learned HL Policies	27

A Summary of Definitions and Notation

We use **Bold Face** font to distinguish low-level (LL) components from high-level (HL) components denoted via *Calligraphic* and *Italics* font. Table 3 summarises the definitions and notation used in the paper.

Table 3: Summary of definitions and notation.

Term	Description	Notation
problem	a tuple of states, actions, initial state and goal states	$\mathbf{P} = \langle \mathbf{S}, \mathbf{A}, \mathbf{s}_0, \mathbf{g} \rangle$
bilevel problem	a LL and HL problem and a labelling function	$\mathbb{P} = \langle \mathbf{P}^{\text{ll}}, \mathbf{P}^{\text{hl}}, \mathcal{L} \rangle$
policy	a probability distribution over actions conditioned on a state and other arguments	$\pi(\mathbf{a} \mid \mathbf{s}, *)$
LL state	low-level object-centric, ego-centric state representation	$\mathbf{s}^{\text{ll}} = (\mathbf{x}_e, \{\mathbf{x}_o\}_{o \in \mathbf{O}})$, $\mathbf{x}_e \in \mathbb{R}^n, \mathbf{x}_o \in \mathbb{R}^m$
LL action	low-level action	$\mathbf{a}^{\text{ll}} \in \mathbb{R}^d$
LL policy	low-level policy	$\pi^{\text{ll}}(\mathbf{a}^{\text{ll}} \mid \mathbf{s}^{\text{ll}}, a^{\text{hl}}, g^{\text{hl}})$
HL state	high-level state consisting of a set of facts	s^{hl}
HL action	high-level action induced by an action schema	$a^{\text{hl}} = a(o_1, \dots, o_{n_a})$
HL policy	high-level policy	$\pi^{\text{hl}}(a^{\text{hl}} \mid s^{\text{hl}}, g^{\text{hl}})$
domain	a set of predicates and action schemata	$\mathcal{D} = \langle \mathcal{P}, \mathcal{A} \rangle$
labelling function	a function mapping LL states to HL states	$\mathcal{L}$
predicate	symbol with argument terms	$p(x_1, \dots, x_{n_p})$
fact	predicate with all arguments instantiated with objects	$p(o_1, \dots, o_{n_p})$
action schema	temporally abstracted action with argument terms	$a(x_1, \dots, x_{n_a})$
dataset	LL demonstrations paired with HL goals	$\mathbb{T} = \{\langle g_i^{\text{hl}}, \mathbf{s}_0^{\text{ll}}, \mathbf{a}_0^{\text{ll}}, \dots, \mathbf{s}_{m_i}^{\text{ll}}, \mathbf{a}_{m_i}^{\text{ll}} \rangle\}_{i=1}^n$
HL dataset	HL action traces paired with HL goals	$T^{\text{hl}} = \{\langle g_i^{\text{hl}}, a_0^{\text{hl}}, \dots, a_{m_i}^{\text{hl}} \rangle\}_{i=1}^n$
succession	the successors of a state under an action	$\text{succ}(\mathbf{s}, \mathbf{a})$
regression	the regressors of a goal under an action	$\text{regr}(\mathbf{g}, \mathbf{a})$

B Theory

We provide additional context and the proof of Theorem 1. The argument requires two ingredients: (1) the HL component of $\mathbb{P}$ admits a solution that proceeds by sequentially achieving singleton goal facts (Section B.1), and (2) $\mathbb{T}$ exhibits enough HL abstracted demonstrations T^{hl} to extract HL rules covering every relevant singleton-goal subproblem (Section B.2).

B.1 Goal Independence

We assume HL problems are represented in the relational form from Section 3. Given an HL problem $\mathbf{P}^{\text{hl}} = \langle \mathbf{S}^{\text{hl}}, \mathbf{A}^{\text{hl}}, s_0^{\text{hl}}, g^{\text{hl}} \rangle$, an HL state $s^{\text{hl}} \in \mathbf{S}^{\text{hl}}$, and a set of facts $g^{\text{hl}'}$, let $\mathbf{P}_{s^{\text{hl}}, g^{\text{hl}'}}^{\text{hl}}$ denote the same HL problem with initial state replaced by s^{hl} and goal replaced by $g^{\text{hl}'}$, i.e. $\mathbf{P}_{s^{\text{hl}}, g^{\text{hl}'}}^{\text{hl}} = \langle \mathbf{S}^{\text{hl}}, \mathbf{A}^{\text{hl}}, s^{\text{hl}}, g^{\text{hl}'} \rangle$. Let $\text{init}(\mathbf{P}^{\text{hl}})$ and $\text{goal}(\mathbf{P}^{\text{hl}})$ denote the initial state and goal of $\mathbf{P}^{\text{hl}}$, respectively.

Definition 2 (Goal Independence). *An HL problem $\mathbf{P}^{\text{hl}}$ exhibits goal independence (GI) if for any ordering $g_1^{\text{hl}}, \dots, g_n^{\text{hl}}$ of the facts in $\text{goal}(\mathbf{P}^{\text{hl}})$, the following procedure constructs a solution for $\mathbf{P}^{\text{hl}}$: set $\pi^{\text{hl}} \leftarrow \emptyset$ and $S \leftarrow \{\text{init}(\mathbf{P}^{\text{hl}})\}$ and for each $i = 1, \dots, n$, (a) extend π^{hl} with solution policies for $\mathbf{P}_{s^{\text{hl}}, \{g_i^{\text{hl}}\}}^{\text{hl}}$ for all $s^{\text{hl}} \in S$ which do not delete any fact g_j^{hl} for $i \neq j$, and (b) add all states in the support of an π_i^{hl} -trajectory ending in a state containing g_i^{hl} to S that are not yet covered by π^{hl} . We further say that $\mathbf{P}^{\text{hl}}$ exhibits C -bounded goal independence (GI_C) for $C \in \mathbb{N}$ if every π_i^{hl} in step (a) has a preimage of size at most C .*

A bilevel planning problem $\mathbb{P} = \langle \mathbf{P}^{\text{ll}}, \mathbf{P}^{\text{hl}}, \mathcal{L} \rangle$ exhibits (resp. C -bounded) goal independence whenever its HL component $\mathbf{P}^{\text{hl}}$ does. The bound C in GI_C ensures each singleton-goal subproblem admits bounded solutions which in turn bounds the size of the rule database extracted by goal regression in Algorithm 1.

B.2 Object-Renaming Equivalence of HL Problems

Goal regression on a single demonstration produces lifted rules that, by virtue of being parameterised by free variables, generalise across object-renaming. We make this precise via an equivalence relation on HL problems.

Definition 3 (Object-Renaming Equivalence). *Two HL problems $\mathbf{P}_1^{\text{hl}}, \mathbf{P}_2^{\text{hl}}$ over the same domain $\mathcal{D}$ are equivalent, written $\mathbf{P}_1^{\text{hl}} \sim \mathbf{P}_2^{\text{hl}}$, if there exists a bijection $f : \mathbf{O}_1 \rightarrow \mathbf{O}_2$ such that the lifted map*

$$F(s^{\text{hl}}) := \{p(f(o_1), \dots, f(o_n)) \mid p(o_1, \dots, o_n) \in s^{\text{hl}}\} \quad (6)$$

satisfies $F(\text{init}(\mathbf{P}_1^{\text{hl}})) = \text{init}(\mathbf{P}_2^{\text{hl}})$ and $F(\text{goal}(\mathbf{P}_1^{\text{hl}})) = \text{goal}(\mathbf{P}_2^{\text{hl}})$.

Proposition 1. *The relation $\sim$ is an equivalence relation. Moreover, suppose $\mathbf{P}_1^{\text{hl}} \sim \mathbf{P}_2^{\text{hl}}$ via a bijection f , and extend F to ground actions by $F(a(o_1, \dots, o_n)) := a(f(o_1), \dots, f(o_n))$. Given any HL policy π_1^{hl} on $\mathbf{P}_1^{\text{hl}}$, define $F_*\pi_1^{\text{hl}}$ on $\mathbf{P}_2^{\text{hl}}$ by*

$$(F_*\pi_1^{\text{hl}})(F(a^{\text{hl}}) \mid F(s^{\text{hl}}), F(g^{\text{hl}})) := \pi_1^{\text{hl}}(a^{\text{hl}} \mid s^{\text{hl}}, g^{\text{hl}}). \quad (7)$$

Then $\sigma = \langle s_0^{\text{hl}}, s_1^{\text{hl}}, \dots \rangle$ is a π_1^{hl} -trajectory in $\mathbf{P}_1^{\text{hl}}$ if and only if $F(\sigma) := \langle F(s_0^{\text{hl}}), F(s_1^{\text{hl}}), \dots \rangle$ is an $(F_\pi_1^{\text{hl}})$ -trajectory in $\mathbf{P}_2^{\text{hl}}$, and consequently π_1^{hl} is a solution for $\mathbf{P}_1^{\text{hl}}$ if and only if $F_*\pi_1^{\text{hl}}$ is a solution for $\mathbf{P}_2^{\text{hl}}$. In particular, a sequence of HL actions $a_0(o_0^1, \dots, o_0^{n_0}), \dots, a_{m-1}(o_{m-1}^1, \dots, o_{m-1}^{n_{m-1}})$ reaches a goal state in $\mathbf{P}_1^{\text{hl}}$ if and only if its image under F reaches a goal state in $\mathbf{P}_2^{\text{hl}}$.*

Proof. Reflexivity, symmetry, and transitivity follow from the analogous properties of bijective functions. For the policy statement, action applicability ($\text{pre}(a^{\text{hl}}) \subseteq s^{\text{hl}}$) and the successor function are determined syntactically by the names of predicates and schemata, which f leaves unchanged; hence F commutes with succ and a^{hl} is applicable in s^{hl} iff $F(a^{\text{hl}})$ is applicable in $F(s^{\text{hl}})$. By construction $F_*\pi_1^{\text{hl}}$ assigns identical support, so π_1^{hl} - and $(F_*\pi_1^{\text{hl}})$ -trajectories are in bijection via F , and goal states correspond by $F(\text{goal}(\mathbf{P}_1^{\text{hl}})) = \text{goal}(\mathbf{P}_2^{\text{hl}})$. The action-sequence claim is the special case where π_1^{hl} is the deterministic open-loop policy that prescribes $a_i(o_i^1, \dots, o_i^{n_i})$ at step i . A consequence of Proposition 1 is that lifted rules extracted from one HL problem transfer verbatim to all problems in the same equivalence class. $\square$

B.3 Generalisation Result

We can now state the formal version of the informal theorem in Section 4. The proof relies on a finite covering argument: under GI_C , the relevant singleton-goal subproblems form a finite collection up to $\sim$, so a finite dataset suffices for the HL rule database to generalise to every test problem.

Theorem 1. *Let $\mathcal{D} = \langle \mathcal{P}, \mathcal{A} \rangle$ be an HL domain, $\mathcal{L}$ a labelling function, and $C \in \mathbb{N}$. There exists a finite dataset $\mathbb{T}$ such that the HL policy learned from $\mathbb{T}$ via Algorithm 1 solves any HL planning problem $\mathbf{P}^{\text{hl}}$ conforming to $\mathcal{D}$ and satisfying C -bounded goal independence (GI_C).*

Proof. Let $N = \max_{a \in \mathcal{A}} |\text{var}(a)|$ and $M = \max_{p \in \mathcal{P}} \text{arity}(p)$ denote the maximum schema and predicate arities, respectively.

Up to $\sim$, the set of singleton-goal HL problems on $\mathcal{D}$ is finite. A singleton goal is a single fact $p(o_1, \dots, o_{n_p})$ which, modulo bijective renaming of objects, is determined by a choice of $p \in \mathcal{P}$ together with an arrangement of its at most M argument positions among at most M distinct objects. There are therefore at most $|\mathcal{P}| \cdot M^M$ singleton goals up to $\sim$. By the GI_C assumption, every singleton-goal subproblem admits a solution policy with size C and hence cycle-less trajectories with size less than C . A length- k HL action sequence together with its singleton goal mentions at most $kN + M$ distinct objects, since each of the k actions has at most N parameter positions and the goal contributes at most M further positions. By Proposition 1 it suffices to count action sequences up to $\sim$: each of the k action positions contributes at most $|\mathcal{A}|$ schema choices and at most $(kN + M)^N$ parameter-tuple choices. Hence the number of inequivalent rules extracted by regressing singleton-goal subproblems through length-at-most- C action sequences is bounded by

$$n \leq |\mathcal{P}| \cdot M^M \cdot \sum_{k=0}^C (|\mathcal{A}| \cdot (kN + M)^N)^k. \quad (8)$$

Choose $\mathbb{T}$ to contain, for each equivalence class of singleton-goal subproblem encountered as a step (a) instance in Definition 2, at least one demonstration whose HL projection realises a solution policy of length at most C for a representative of that class. Such a $\mathbb{T}$ is finite by Equation (8).

Now fix any HL problem $\mathbf{P}^{\text{hl}}$ satisfying the theorem assumptions and let $g_1^{\text{hl}}, \dots, g_n^{\text{hl}}$ be any goal ordering. Let π^{hl} denote the HL policy synthesised from $\mathbb{T}$ by Algorithm 1. By construction, π^{hl} is the union of lifted rules obtained by goal regression of the demonstrations in $\mathbb{T}$. By Step 1 and Proposition 1, for every HL state s^{hl} reached during execution and every singleton subgoal g_i^{hl} not yet achieved in s^{hl} , there exists a grounded rule $r \in \text{ground}(\pi^{\text{hl}})$ that is applicable in state s^{hl} with goal g_i^{hl} . Executing $\text{action}(r)$ thus prescribes a valid HL action that progresses $\mathbf{P}_{s^{\text{hl}}, \{g_i^{\text{hl}}\}}^{\text{hl}}$ and reduces the value of the next applicable rule in the HL policy. Iterating the construction over the n goal facts yields a solution for $\mathbf{P}^{\text{hl}}$ by definition of GI. $\square$

The bound Equation (8) is exponential in the horizon C and the maximum schema arity N , mirroring the worst-case complexity of generalised planning over relational domains. In practice, the number of demonstrations required is far smaller, since most singleton-goal subproblems share regression-extracted rules that lift to many equivalence classes simultaneously, and our experiments in Section 5 confirm that a modest training set already suffices for π^{hl} to generalise to arbitrary numbers of objects.

C Additional Experimental Details

C.1 GNN Hyperparameters

We train all GNN models using the Adam optimiser [65] for 200 iterations with an initial learning rate of 10^{-3} , batch size of 128, and a cosine annealing learning rate scheduler [86]. Each GNN has a hidden and output dimension of 64 and 2 message passing layers.

C.2 Baselines

We provide more details on the baselines of our experiments.

SmolVLA SmolVLA is the 450 million parameter open-source SmolVLA [117], accessed from https://huggingface.co/lerobot/smolvla_base

SmolVLA^{MW} SmolVLA^{MW} is SmolVLA that is finetuned on 2500 episodes of the original MetaWorld benchmark suite (50 episodes from each of the 50 tasks), accessed from https://huggingface.co/jadechoghari/smolvla_metaworld

PureNN PureNN is a special case of PddlNN in which all HL information is not encoded by assuming that $s^{\text{hl}} = g^{\text{hl}} = \emptyset$ and by not encoding an HL action node.

PddlNN PddlNN is a similar GNN architecture to the one in Section 4.2 which encodes all HL state information including all facts and all other objects but no HL action. We can view PddlNN as a purely end-to-end approach for handling bilevel planning that aims to learn the HL policy in place of using a separately learned, symbolic HL policy. We sketch the heterogeneous GNN architecture for PddlNN as follows.

The inputs are $s^{\text{ll}}, g^{\text{hl}}$ and $s^{\text{hl}} = \mathcal{L}(s^{\text{ll}})$ and encoded as a graph with a global node with initial embedding $\mathbf{h}_{\text{global}}$, no action node, object nodes with embeddings $\{\mathbf{h}_o \mid o \in \mathbf{O}\}$ and fact nodes with embeddings $\{\mathbf{h}_f \mid f \in s^{\text{hl}} \cup g^{\text{hl}}, \text{arity}(f) \geq 2\}$. The initial embeddings are now given by

- $\mathbf{h}_{\text{global}} = \mathbf{x}_e \parallel \sum_{p() \in s^{\text{hl}}} \mathbf{e}_p^{|\mathcal{P}|} \parallel \sum_{p() \in g^{\text{hl}}} \mathbf{e}_p^{|\mathcal{P}|}$
- $\mathbf{h}_o = \mathbf{x}_o \parallel \sum_{p(o) \in s^{\text{hl}}} \mathbf{e}_p^{|\mathcal{P}|} \parallel \sum_{p(o) \in g^{\text{hl}}} \mathbf{e}_p^{|\mathcal{P}|}$ and
- $\mathbf{h}_f = [f \in s^{\text{hl}}, f \in g^{\text{hl}}] \parallel \mathbf{e}_p^{|\mathcal{P}|}$ where p is the predicate of f .

The GNN algorithm for PddlNN then consists of the following three operations. All $\mathbf{W}$ symbols denote learnable weights.

- The initial encodings are embedded $\mathbf{h}_g^{(0)} = \mathbf{W}_g^{(0)} \mathbf{h}_g$, $\mathbf{h}_o^{(0)} = \mathbf{W}_o^{(0)} \mathbf{h}_o$, and $\mathbf{h}_f^{(0)} = \mathbf{W}_f^{(0)} \mathbf{h}_f$.

- (b) Next, L rounds of heterogeneous message passing¹ are performed. Let $\langle U, e, V \rangle$ denote a directed edge type from node type U to node type V via relation e . We overload the notations U and $\langle U, e, V \rangle$ to also mean a set that contains the nodes of node type U , and pairs of nodes $\langle u, v \rangle$ which have an edge of type $\langle U, e, V \rangle$, respectively. For each fact node $f = p(o_1, \dots, o_m) \in s^{\text{hl}} \cup g^{\text{hl}}$ with $m \geq 2$, we include bidirectional edge types $\langle F, \text{fct}_j, O \rangle$ and $\langle O, \text{fct}_j^{-1}, F \rangle$ for each $j = 1, \dots, m$. Node embeddings are updated per edge type by aggregating neighbour messages and the global node to give the updated embedding

$$\mathbf{h}_v^{(l+1)} = \sigma \left(\mathbf{W}_v \mathbf{h}_v^{(l)} + \mathbf{W}_{\text{global}} \mathbf{h}_{\text{global}}^{(l)} + \sum_{\langle U, e, V \rangle} \max_{\langle u, v \rangle \in \langle U, e, V \rangle} \mathbf{W}_{\langle U, e, V \rangle}^{(l)} \mathbf{h}_u^{(l)} \right)$$

for nonlinearity σ . We then update the global node

$$\mathbf{h}_{\text{global}}^{(l+1)} = \sigma \left(\sum_U \max_{u \in U} \mathbf{W}_U \mathbf{h}_u^{(l+1)} \right).$$

- (c) Finally, a feed-forward readout layer is applied to the sum of all final embeddings to predict the LL action $\mathbf{a}^{\text{ll}}$.

DetPlan Algorithm 2 illustrates the DetPlan baseline in more detail. It firstly computes an HL plan $a_1, \dots, a_n$ (Algorithm 2) and aims to naively and faithfully follow it throughout the simulation. If the precondition of the current HL action, which is tracked by the index i , is satisfied in the current HL state (Algorithm 2), then we keep it. Otherwise we check if the next action exists and its precondition is satisfied (Algorithm 2). Otherwise, we determine that the HL plan is broken and terminate with failure as no HL action can be computed (Algorithm 2).

Algorithm 2: DetPlan Baseline

Input: Problem $\mathbf{P}$ domain $\mathcal{D}$, labelling function $\mathcal{L}$, HL goal g^{hl} , and LL policy π^{ll} .

```

1  $\mathbf{s}^{\text{ll}} \leftarrow \mathbf{s}_0$ 
2  $s^{\text{hl}} \leftarrow \mathcal{L}(\mathbf{s}^{\text{ll}})$ 
3  $a_1, \dots, a_n \leftarrow \text{findPlan}(s^{\text{hl}}, g^{\text{hl}})$ 
4  $i \leftarrow 1$ 
5 while  $s^{\text{hl}} \not\subseteq g^{\text{hl}}$  and  $i \leq n$ 
6    $s^{\text{hl}} \leftarrow \mathcal{L}(\mathbf{s}^{\text{ll}})$ 
7   if  $\text{pre}(a_i) \subseteq s^{\text{hl}}$  then
8      $a^{\text{hl}} \leftarrow a_i$ 
9   else if  $\text{pre}(a_i) \not\subseteq s^{\text{hl}}$  and  $i + 1 \leq n$  and  $\text{pre}(a_{i+1}) \subseteq s^{\text{hl}}$  then
10     $i \leftarrow i + 1$ 
11     $a^{\text{hl}} \leftarrow a_i$ 
12   else
13     return Failure
14    $\mathbf{a}^{\text{ll}} \leftarrow \pi^{\text{ll}}(\mathbf{s}^{\text{ll}}, a^{\text{hl}}, g^{\text{hl}})$ 
15    $\mathbf{s}^{\text{ll}} \leftarrow \text{exec}(\mathbf{s}^{\text{ll}}, \mathbf{a}^{\text{ll}})$ 
```

DetReplan Algorithm 3 illustrates the DetReplan baseline in more detail. It is the same as Algorithm 2 except in how it handles the failure case (Algorithm 3) by replanning. It computes a new HL plan and resets the index i and length of the plan n .

NdtPlan

Algorithm 4 illustrates the NdtPlan baseline in more detail. It firstly aims to compute an HL policy specific to the problem and tries to execute it. The major difference is that the HL policy may return nothing for a given state, in which case we terminate with failure.

NdtReplan Algorithm 5 illustrates the NdtReplan baseline in more detail. It is the same as Algorithm 4 except it now aims to replan by recomputing an HL policy for the failure case (Algorithm 3).

C.3 Benchmarks

The benchmarks compositionally extend the MetaWorld [147, 93] benchmarks which consist of 3D MuJoCo simulations of a robot arm. The following are descriptions of the environment tasks and

¹<https://pytorch-geometric.readthedocs.io/en/latest/tutorial/heterogeneous.html>

Algorithm 3: DetReplan Baseline

Input: Problem $\mathbf{P}$ domain $\mathcal{D}$, labelling function $\mathcal{L}$, HL goal g^{hl} , and LL policy π^{ll} .

```
1  $s^{\text{ll}} \leftarrow s_0$ 
2  $s^{\text{hl}} \leftarrow \mathcal{L}(s^{\text{ll}})$ 
3  $a_1, \dots, a_n \leftarrow \text{findPlan}(s^{\text{hl}}, g^{\text{hl}})$ 
4  $i \leftarrow 1$ 
5 while  $s^{\text{hl}} \notin g^{\text{hl}}$  and  $i \leq n$ 
6    $s^{\text{hl}} \leftarrow \mathcal{L}(s^{\text{ll}})$ 
7   if  $\text{pre}(a_i) \subseteq s^{\text{hl}}$  then
8      $a^{\text{hl}} \leftarrow a_i$ 
9   else if  $\text{pre}(a_i) \not\subseteq s^{\text{hl}}$  and  $i + 1 \leq n$  and  $\text{pre}(a_{i+1}) \subseteq s^{\text{hl}}$  then
10     $i \leftarrow i + 1$ 
11     $a^{\text{hl}} \leftarrow a_i$ 
12  else
13     $a_1, \dots, a_n \leftarrow \text{findPlan}(s^{\text{hl}}, g^{\text{hl}})$ 
14     $i \leftarrow 1$ 
15    continue
16   $a^{\text{ll}} \leftarrow \pi^{\text{ll}}(s^{\text{ll}}, a^{\text{hl}}, g^{\text{hl}})$ 
17   $s^{\text{ll}} \leftarrow \text{exec}(s^{\text{ll}}, a^{\text{ll}})$ 
```

Algorithm 4: NdtPlan Baseline

Input: Problem $\mathbf{P}$ domain $\mathcal{D}$, labelling function $\mathcal{L}$, HL goal g^{hl} , and LL policy π^{ll} .

```
1  $s^{\text{ll}} \leftarrow s_0$ 
2  $s^{\text{hl}} \leftarrow \mathcal{L}(s^{\text{ll}})$ 
3  $\pi^{\text{hl}} \leftarrow \text{findPolicy}(s^{\text{hl}}, g^{\text{hl}})$ 
4 while  $s^{\text{hl}} \notin g^{\text{hl}}$ 
5    $s^{\text{hl}} \leftarrow \mathcal{L}(s^{\text{ll}})$ 
6    $a^{\text{hl}} \leftarrow \pi^{\text{hl}}(s^{\text{hl}}, g^{\text{hl}})$ 
7   if  $a^{\text{hl}} = \perp$  then
8     return Failure
9    $a^{\text{ll}} \leftarrow \pi^{\text{ll}}(s^{\text{ll}}, a^{\text{hl}}, g^{\text{hl}})$ 
10   $s^{\text{ll}} \leftarrow \text{exec}(s^{\text{ll}}, a^{\text{ll}})$ 
```

also serve as the natural language inputs to the VLA baseline. The environments are illustrated in Figure 7.

Blocks^S Place the n blocks to goal locations of the same colour.

Blocks^N Place the n blocks to goal locations of the same colour. Blocks that are currently at their goal location may teleport to an unoccupied location at any time.

Factory^S Place the n blocks to goal locations of the same colour. Every time each of the original blocks is placed, a new block is introduced with a new goal location.

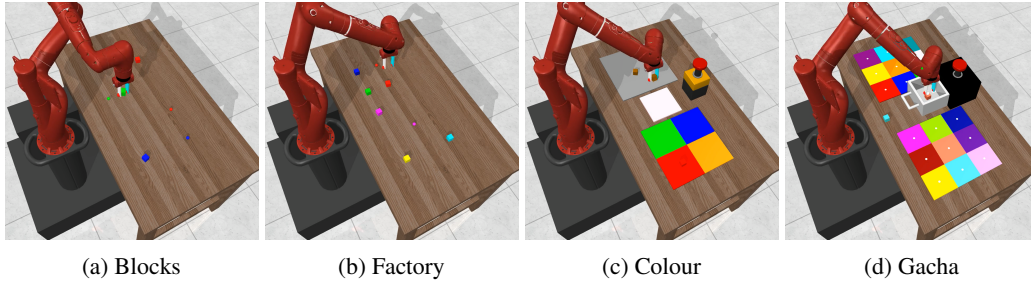


Figure 7: Visualisations of benchmark environments.

Algorithm 5: NdtReplan Baseline

Input: Problem $\mathbf{P}$ domain $\mathcal{D}$, labelling function $\mathcal{L}$, HL goal g^{hl} , and LL policy π^{ll} .

```
1  $s^{\text{ll}} \leftarrow s_0$ 
2  $s^{\text{hl}} \leftarrow \mathcal{L}(s^{\text{ll}})$ 
3  $\pi^{\text{hl}} \leftarrow \text{findPolicy}(s^{\text{hl}}, g^{\text{hl}})$ 
4 while  $s^{\text{hl}} \notin g^{\text{hl}}$ 
5    $s^{\text{hl}} \leftarrow \mathcal{L}(s^{\text{ll}})$ 
6    $a^{\text{hl}} \leftarrow \pi^{\text{hl}}(s^{\text{hl}}, g^{\text{hl}})$ 
7   if  $a^{\text{hl}} = \perp$  then
8      $\pi^{\text{hl}} \leftarrow \text{findPolicy}(s^{\text{hl}}, g^{\text{hl}})$ 
9     continue
10   $a^{\text{ll}} \leftarrow \pi^{\text{ll}}(s^{\text{ll}}, a^{\text{hl}}, g^{\text{hl}})$ 
11   $s^{\text{ll}} \leftarrow \text{exec}(s^{\text{ll}}, a^{\text{ll}})$ 
```

Factory^N Place the n blocks to goal locations of the same colour. Every time each of the original blocks is placed, a new block is introduced with a new goal location. Blocks that are currently at their goal location may also teleport to an unoccupied location at any time.

Colour^S Place the n brown blocks onto the pink platform, which assigns blocks a random colour when activated. The goal is to colour each of the brown blocks and then place them on a tray with the same colour.

Colour^N Place the n brown blocks onto the pink platform, which assigns blocks a random colour when activated. The goal is to colour each of the brown blocks and then place them on a tray with the same colour. Blocks that are currently at their goal location may also teleport to an unoccupied location at any time.

Gacha^S This environment consists of n target colours, a gallery of trays with different colours, and a *gacha* box which when its lid is closed and activated produces a block of a random colour. The goal is to match each target colour by placing a block of that colour on its corresponding tray.

Gacha^N This environment consists of n target colours, a gallery of trays with different colours, and a *gacha* box which when its lid is closed and activated produces a block of a random colour. The goal is to match each target colour by placing a block of that colour on its corresponding tray. Blocks that are currently at their goal location may teleport back into the gacha box whenever it is unoccupied.

C.4 Visualising Learned HL Policies

We display the learned HL policies for each of the environments which can generalise to arbitrary numbers of objects. We refer to Example 2 for interpreting the form of the rules. We automatically generate LLM interpretations of the policies via Claude Sonnet 4.6.

Blocks^S and Blocks^N

$$\begin{aligned} 1 : \exists x, l. \text{holding}(x), \text{clear}(l), \underline{\text{at}(x, l)} &\mapsto \text{place}(x, l) \\ 2 : \exists x, l. \text{clear}(x), \text{gripperFree}(), \underline{\text{at}(x, l)} &\mapsto \text{pick}(x) \end{aligned}$$

LLM-interpretation: The policy has two rules, applied by priority. Rule 1 says: if the gripper is holding block x and its goal location l is clear, place x at l . Rule 2 says: if block x is clear (nothing on top), the gripper is free, and x has a goal location, pick up x . Together these encode a simple pick-and-place loop: pick the next block with an unmet goal, then immediately place it. The policy also handles Blocks^N reactively, since the goal predicate $\text{at}^G(x, l)$ remains true for any block teleported away from its goal, so it gets re-picked on the next step.

Factory^S and Factory^N

$$\begin{aligned} 1 : \exists x, l. \text{holding}(x), \text{clear}(l), \underline{\text{at}(x, l)} &\mapsto \text{place}(x, l) \\ 2 : \exists x, l. \text{clear}(x), \text{gripperFree}(), \underline{\text{at}(x, l)} &\mapsto \text{pick}(x) \end{aligned}$$

LLM-interpretation: The rules are identical to those of Blocks^S. This is notable: even though new blocks are introduced dynamically when existing ones are placed, the policy generalises automatically because its rules are existentially quantified over objects. Any newly introduced block x with goal location l immediately satisfies Rule 2 and gets picked, then placed by Rule 1, with no change to the policy itself.

Colour^S and Colour^N

$$\begin{aligned} 1 : \exists c, x, t. \text{colourOf}(x, c), \text{colourOf}(t, c), \text{holding}(x), \underline{\text{achievedGoal}(x)} &\mapsto \text{placeGoal}(x, t, c) \\ 2 : \exists c, x, t. \text{colourOf}(x, c), \text{colourOf}(t, c), \text{gripperFree}(), \text{on}(x, t), \underline{\text{achievedGoal}(x)} &\mapsto \text{pick}(x) \\ 2 : \exists c, x, t. \text{colourOf}(x, c), \text{colourOf}(t, c), \text{gripperFree}(), \underline{\text{achievedGoal}(x)} &\mapsto \text{pick}(x) \\ 3 : \exists b, c, x, t, t'. \text{colourOf}(t, c), \text{colourer}(t'), \text{gripperFree}(), \text{on}(x, t'), \text{unknown}(x), \underline{\text{achievedGoal}(x)} &\mapsto \text{open}(x, b, t') \\ 4 : \exists c, x, t, t'. \text{colourOf}(t, c), \text{colourerFree}(), \text{colourer}(t'), \text{holding}(x), \text{unknown}(x), \underline{\text{achievedGoal}(x)} &\mapsto \text{place}(x, t') \\ 5 : \exists c, x, t, t'. \text{colourOf}(t, c), \text{colourer}(t'), \text{gripperFree}(), \text{on}(x, t'), \text{unknown}(x), \underline{\text{achievedGoal}(x)} &\mapsto \text{pick}(x) \\ 5 : \exists c, x, t, t'. \text{colourOf}(t, c), \text{colourerFree}(), \text{colourer}(t'), \text{gripperFree}(), \text{unknown}(x), \underline{\text{achievedGoal}(x)} &\mapsto \text{pick}(x) \end{aligned}$$

LLM-interpretation: The task requires colouring brown (unknown) blocks on a pink platform before placing them on matching trays, and the policy decomposes the workflow into five priority levels. Rule 1: if holding a block x whose colour c matches a tray t of the same colour, place x on t . Rule 2 (two variants): if a coloured block x already matches a tray colour, pick it up; the two variants differ only in whether x is currently resting on a surface. Rule 3: if an unknown block x is sitting on the colourer t' , activate the colourer (*open*) to assign x a colour. Rule 4: if the gripper is holding an unknown block and the colourer is free, place the block on the colourer. Rule 5 (two variants): if a block has no colour yet, pick it up so it can be brought to the colourer. The priority ordering encodes the full pipeline: fetch unknown block, place it on the colourer, activate the colourer, then pick up the now-coloured block and place it on the matching tray. The same policy handles Colour^N reactively, as in Blocks^S.

Gacha^S and Gacha^N

$$\begin{aligned}
& 1 : \exists c, x, t. \text{ colourOf}(x, c), \text{ holding}(x), \text{ trayColour}(t, c), \underline{\text{achievedGoal}(c)} \\
& \quad \mapsto \text{placeGoal}(x, t, c) \\
& 2 : \exists c, x, d, t. \text{ colourOf}(x, c), \text{ gripperFree}(), \text{ in}(x, d), \text{ opened}(d), \text{ trayColour}(t, c), \underline{\text{achievedGoal}(c)} \\
& \quad \mapsto \text{pick}(x) \\
& 3 : \exists c, x, d, t. \text{ colourOf}(x, c), \text{ gripperFree}(), \text{ in}(x, d), \text{ trayColour}(t, c), \underline{\text{achievedGoal}(c)} \\
& \quad \mapsto \text{open}(d) \\
& 4 : \exists b, c, d, t. \text{ clear}(d), \text{ closed}(d), \text{ gripperFree}(), \text{ trayColour}(t, c), \underline{\text{achievedGoal}(c)} \\
& \quad \mapsto \text{roll}(d, b) \\
& 5 : \exists c, d, t. \text{ clear}(d), \text{ gripperFree}(), \text{ opened}(d), \text{ trayColour}(t, c), \underline{\text{achievedGoal}(c)} \\
& \quad \mapsto \text{close}(d) \\
& 6 : \exists c, c', x, d, t, t'. \text{ clear}(d), \text{ colourOf}(x, c), \text{ holding}(x), \text{ opened}(d), \text{ trayColour}(t, c), \text{ trayColour}(t', c'), \underline{\text{achievedGoal}(c')} \\
& \quad \mapsto \text{placeGoal}(x, t, c) \\
& 7 : \exists c, c', x, d, t, t'. \text{ colourOf}(x, c), \text{ gripperFree}(), \text{ in}(x, d), \text{ opened}(d), \text{ trayColour}(t, c), \text{ trayColour}(t', c'), \underline{\text{achievedGoal}(c')} \\
& \quad \mapsto \text{pick}(x) \\
& 7 : \exists c, c', x, d, t, t'. \text{ colourOf}(x, c), \text{ gripperFree}(), \text{ in}(x, d), \text{ opened}(d), \text{ trayColour}(t, c'), \text{ trayColour}(t', c), \underline{\text{achievedGoal}(c')} \\
& \quad \mapsto \text{pick}(x) \\
& 8 : \exists c, c', x, d, t, t'. \text{ colourOf}(x, c), \text{ gripperFree}(), \text{ in}(x, d), \text{ trayColour}(t, c'), \text{ trayColour}(t', c), \underline{\text{achievedGoal}(c')} \\
& \quad \mapsto \text{open}(d) \\
& 8 : \exists c, c', x, d, t, t'. \text{ colourOf}(x, c), \text{ gripperFree}(), \text{ in}(x, d), \text{ trayColour}(t, c), \text{ trayColour}(t', c'), \underline{\text{achievedGoal}(c')} \\
& \quad \mapsto \text{open}(d) \\
& 9 : \exists b, c, c', d, t, t'. \text{ clear}(d), \text{ closed}(d), \text{ gripperFree}(), \text{ trayColour}(t, c), \text{ trayColour}(t', c'), \underline{\text{achievedGoal}(c)} \\
& \quad \mapsto \text{roll}(d, b) \\
& 9 : \exists b, c, c', d, t, t'. \text{ clear}(d), \text{ closed}(d), \text{ gripperFree}(), \text{ trayColour}(t, c), \text{ trayColour}(t', c'), \underline{\text{achievedGoal}(c')} \\
& \quad \mapsto \text{roll}(d, b)
\end{aligned}$$

LLM-interpretation: The task requires producing blocks of specific colours from a gacha box (close lid, roll, open lid, then extract block) and placing them on matching trays. The policy has nine priority levels. Rule 1: if holding a block x of the needed colour c , place it on the matching tray t . Rule 2: if the gacha d is open and contains a block of the needed colour, pick it. Rule 3: if the gacha contains a block of the needed colour but is closed, open it. Rule 4: if the gacha is closed and empty, roll it to produce a new block of random colour. Rule 5: if the gacha is open but does not contain a useful block, close it so that it can be rolled again. Rules 6–8 cover *opportunistic* actions when the currently pursued goal colour is c' but a block of a different needed colour c becomes available: place it on its matching tray (Rule 6), pick it from an open gacha (Rule 7, two variants differing in the binding of c and c' to the trays), or open the gacha to retrieve it (Rule 8, two variants). Rule 9 (two variants) rolls the gacha when at least two unachieved goal colours remain and the gacha is empty. The policy is more complex than Blocks^S because it must reason about both the stochastic production mechanism (rolls produce random colours) and opportunity costs across multiple simultaneous colour goals; Rules 6–9 in particular ensure that lucky rolls are never wasted, even when they do not match the colour the agent is currently chasing.